

Politechnika Śląska

PRACA MAGISTERSKA

Zintegrowany system percepcji i nawigacji dla robota mobilnego w środowisku terenowym

Maksymilian GRABOWY
Nr albumu: 300365

Kierunek: Automatyka i Robotyka
Specjalność: Robotyka

PROWADZĄCY PRACĘ
dr inż. Krzysztof Jaskot
KATEDRA Automatyki i Robotyki
Wydział Automatyki, Elektroniki i Informatyki

Gliwice 2026

Tytuł pracy

Zintegrowany system percepcji i nawigacji dla robota mobilnego w środowisku terenowym

Streszczenie

(Streszczenie pracy – odpowiednie pole w systemie APD powinno zawierać kopię tego streszczenia.)

Słowa kluczowe

(2-5 slow (fraz) kluczowych, oddzielonych przecinkami)

Thesis title

Thesis title in English

Abstract

(Thesis abstract – to be copied into an appropriate field during an electronic submission – in English.)

Key words

(2-5 keywords, separated by commas)

Spis treści

1 Wprowadzenie **1**
1.1 Wstęp i motywacja 1
1.2 Cel i zakres pracy 1

2 Przegląd literatury **3**
2.1 Robotyka mobilna w zastosowaniach terenowych 3
2.2 Systemy percepcji w robotyce terenowej 4
2.3 Metody lokalizacji i mapowania 4
2.4 Metody nawigacji 6
2.4.1 Algorytmy planowania ruchu 7
2.5 Nowoczesne architektury oprogramowania w robotyce 8
2.5.1 Paradygmat Drzew Zachowań 9

3 Wstęp teoretyczny **11**
3.1 Przestrzenny model kinematyczny 11
3.1.1 Ograniczenia modeli kinematycznych w środowisku terenowym . . . 12
3.2 Technologie sensoryczne w robotyce terenowej 13
3.2.1 Skanery laserowe 13
3.2.2 Kamery głębi oparte na pomiarze czasu przelotu (ToF) 16
3.2.3 Inercyjne jednostki pomiarowe (IMU) 17
3.3 Teoria estymacji stanu i fuzji danych sensorycznych 18
3.4 Algorytmy SLAM 20
3.4.1 Rejestracja chmur punktów 20
3.4.2 Organizacja estymacji i optymalizacja trajektorii 21
3.4.3 Domykanie pętli i reprezentacja mapy 22
3.4.4 Ograniczenia w środowisku terenowym 22
3.5 Modele planowania ruchu i omijania przeszkód 23
3.5.1 Reprezentacja terenu i mapy kosztów 23
3.5.2 Planowanie globalne i kryterium kosztu 24
3.5.3 Śledzenie ścieżki i sterowanie lokalne 24
3.5.4 Reakcja na przeszkody i ograniczenia terenowe 25

4 Architektura systemu i platformy sprzętowe **27**
4.1 Autonomiczny robot terenowy Rebob 27
4.2 Architektura oprogramowania 27
4.3 Standardy tworzenia oprogramowania i przygotowanie systemu do wdrożeń 27

5 Środowisko symulacyjne **29**
5.1 Modelowanie środowiska symulacyjnego w symulatorze Gazebo 29
5.2 Tworzenie modeli kinematycznych i dynamicznych 32
5.3 Symulacja systemów percepcyjnych 34
5.3.1 Skaner laserowy (LIDAR 3D) 34
5.3.2 Kamera RGB-D 36
5.3.3 Inercyjna jednostka pomiarowa (IMU) 38

6 Opracowanie zintegrowanego systemu percepcji **39**
6.1 Integracja czujników 39
6.2 Przetwarzanie chmur punktów i filtracja danych 39
6.3 Implementacja metod 3D SLAM 39
6.4 Analiza przejezdności terenu 39

7 Konfiguracja i implementacja metod nawigacyjnych **41**
7.1 Konfiguracja przestrzeni kosztów dla środowiska 3D 41
7.2 Implementacja metod nawigacyjnych 41
7.3 Adaptacja metod nawigacyjnych do platform testowych 41

8 Badania i analiza wyników **43**
8.1 Metodyka badań i wskaźniki oceny wydajności 43
8.2 Eksperymenty w środowisku symulacyjnym 43
8.3 Testy rzeczywiste algorytmów na platformach testowych 43
8.4 Porównanie metod 3D SLAM 43
8.5 Porównanie algorytmów planowania ruchu 43
8.6 Wpływ kinematyki na skuteczność nawigacji terenowej 43

9 Podsumowanie i wnioski **45**
9.1 Podsumowanie zrealizowanych prac 45
9.2 Ocena uzyskanych wyników 45
9.3 Perspektywy dalszego rozwoju 45

Bibliografia **47**

Dokumentacja techniczna **57**

Spis skrótów i symboli **59**

Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy) **61**

Spis rysunków **63**

Spis tabel **65**

Rozdział 1

Wprowadzenie

1.1 Wstęp i motywacja

W ostatnich latach obserwuje się dynamiczny rozwój systemów autonomicznych. Roboty mobilne, wykorzystywane wcześniej głównie w laboratoriach badawczych i kontrolowanych środowiskach przemysłowych, coraz częściej realizują zadania w rolnictwie, ratownictwie, górnictwie oraz podczas eksploracji obszarów trudno dostępnych [24].

Duża część robotów mobilnych, szczególnie w dziedzinie intralogistyki, porusza się tylko i wyłącznie w tak zwanych środowiskach ustrukturyzowanych - na płaskich posadzkach hal magazynowych czy produkcyjnych, gdzie warunki oświetleniowe są stałe, a trasy przejazdu przewidywalne. Zasadniczym problemem w robotyce mobilnej pozostaje jednak nawigacja w środowiskach nieustrukturyzowanych i terenowych jako uniwersalne narzędzie dla pojazdów o różnym układzie kinematycznym. Nierówności podłoża, dynamicznie pojawiające się przeszkody oraz brak punktów odniesienia sprawiają że standardowe metody lokalizacji i planowania ruchu stają się niewystarczające.

Aby robot mógł skutecznie operować w takich warunkach, niezbędne jest wyposażenie go w zaawansowane systemy percepcji zdolne do fuzji danych z wielu źródeł. Właściwa interpretacja tych danych w czasie rzeczywistym jest kluczowa dla bezpiecznego planowania ruchu. Ponadto, rosnąca złożoność algorytmów wymusza stosowanie nowoczesnych, rozproszonych architektur oprogramowania. Z tego względu, motywacją do podjęcia niniejszej pracy jest potrzeba opracowania niezawodnego systemu autonomii dla platformy mobilnej operującej w terenie, z wykorzystaniem nowoczesnego ekosystemu ROS2, który wyznacza obecnie standardy w rozwoju robotyki.

1.2 Cel i zakres pracy

Głównym celem pracy jest zaprojektowanie, implementacja i weryfikacja modułowego systemu percepcji i nawigacji przeznaczonego dla robotów mobilnych operujących w śro-

dowisku terenowym. Przenośność rozwiązania zweryfikowano z wykorzystaniem platform o odmiennej konstrukcji i sposobie poruszania się.

Osiągnięcie postawionego celu wymagało rozwiązania szeregu problemów natury algorytmicznej oraz integracyjnej. Z tego względu, w zakres zrealizowanych prac wchodzą następujące zadania:

- Opracowanie wirtualnego modelu kinematycznego robota oraz zdefiniowanie charakterystyk jego systemów percepcyjnych w symulatorze środowiskowym Gazebo.
- Przygotowanie zróżnicowanych scenariuszy testowych w postaci modeli środowisk terenowych uwzględniających nierówności podłoża oraz przeszkody naturalne.
- Implementacja mechanizmów fuzji danych z czujników zamontowanych na robocie.
- Konfiguracja i uruchomienie algorytmów jednoczesnego mapowania i lokalizacji w przestrzeni trójwymiarowej.
- Zaprojektowanie stosu nawigacyjnego w systemie ROS2, odpowiedzialnego za globalne i lokalne planowanie ruchu pojazdu z uwzględnieniem trójwymiarowej geometrii otoczenia oraz dynamicznego omijania przeszkód.
- Przeprowadzenie badań weryfikujących poprawność i skuteczność zaproponowanych rozwiązań.

Praca została podzielona na 9 rozdziałów. Po niniejszym wprowadzeniu, w rozdziale drugim omówiono ramy teoretyczne... [tu w przyszłości krótko opiszę strukturę pracy].

Rozdział 2

Przegląd literatury

2.1 Robotyka mobilna w zastosowaniach terenowych

Znaczna część współczesnych wdrożeń dotyczy środowisk terenowych, w których roboty mobilne realizują zadania o podwyższonym stopniu ryzyka lub wymagające dużej precyzji w trudnych warunkach. Do głównych obszarów ich wykorzystania należą:

- **Rolnictwo precyzyjne** - autonomiczne ciągniki i platformy monitorujące uprawy [36],
- **Leśnictwo** - bezzałogowe statki powietrzne (UAV) i platformy lądowe wykrywające zalążki pożarów [56],
- **Przemysł wydobywczy** - roboty inspekcyjne monitorujące skład atmosfery oraz stabilność wyrobisk w kopalniach [44],
- **Ratownictwo (Search and Rescue)** - platformy wspomagające poszukiwania w obszarach dotkniętych katastrofami naturalnymi [10],
- **Obronność** - taktyczne roboty wsparcia i platformy patrolowe [59],
- **Eksploracja kosmiczna** - łaziki planetarne operujące w warunkach całkowitego braku infrastruktury.

Środowiska terenowe definiowane są jako obszary wysoce nieustrukturyzowane. Oznacza to brak założeń upraszczających, takich jak obecność płaskich posadzek, prostopadłych ścian czy jednoznacznych linii krawędziowych, na których opierają się systemy intralogistyczne. Nawigacja w takich warunkach jest dodatkowo utrudniona przez zmienne czynniki środowiskowe (kurz, mgła, deszcz) oraz drastyczne wahania oświetlenia, prowadzące do degradacji danych sensorycznych. Przykładowo, gęsty pył wzbijany przez koła platformy mobilnej może być błędnie interpretowany przez skanery laserowe jako fizyczna przeszkoda, podczas gdy silne nasłonecznienie doprowadza do prześwietlenia matryc kamer, uniemożliwiając poprawną ekstrakcję cech terenu. Wymusza to stosowanie zaawansowanych i redundantnych systemów percepcji.

2.2 Systemy percepcji w robotyce terenowej

Skuteczna nawigacja w terenie opiera się na integracji komplementarnych technologii sensorycznych. Klasyczne podejścia, bazujące wyłącznie na odometrii kół, ulegają w terenie szybkiej degradacji na skutek poślizgów na luźnym lub grząskim podłożu [12]. W związku z tym, współczesne systemy autonomiczne integrują dane pozyskiwane z sensorów otoczenia (LIDAR, systemy wizyjne, GNSS) z pomiarami stanu wewnętrznego robota (IMU).

Analiza dostępnych rozwiązań wskazuje, że systemy percepcji robotów terenowych są najczęściej projektowane jako układy wielosensorowe. Poszczególne źródła danych nie są traktowane jako rozwiązania konkurencyjne, lecz jako wzajemnie uzupełniające się elementy systemu. Skanery laserowe wykorzystywane są przede wszystkim do odwzorowania geometrii otoczenia i dopasowywania kolejnych skanów, jednostki inercyjne zapewniają pomiary ruchu o wysokiej częstotliwości, natomiast systemy wizyjne dostarczają informacji fotometrycznych i semantycznych. Pomiary GNSS mogą dodatkowo ograniczać długookresowy dryft estymacji w środowiskach otwartych. Szczególnie istotną grupę stanowią systemy odometrii laserowo-inercyjnej, w których dane z IMU wykorzystywane są do predykcji ruchu i kompensacji zniekształceń chmury punktów, natomiast pomiary skanera ograniczają narastanie błędów estymacji [55].

Zestaw wykorzystywanych czujników powinien być dobierany z uwzględnieniem warunków pracy robota. Kamery są podatne na zmiany oświetlenia, skanery laserowe na rozpraszanie promieniowania przez opady i pył, GNSS może być niedostępny w pobliżu zabudowań lub pod koronami drzew, a estymacja oparta wyłącznie na IMU podlega szybkiemu dryftowi. Z tego względu w literaturze dotyczącej robotyki terenowej dominuje podejście oparte na fuzji danych, którego celem jest wykorzystanie komplementarnych właściwości poszczególnych modalności sensorycznych [51, 60, 6].

2.3 Metody lokalizacji i mapowania

Jednym z podstawowych problemów robotyki mobilnej jest jednoczesna lokalizacja i budowanie mapy (SLAM, ang. *Simultaneous Localization and Mapping*). Zadaniem systemu SLAM jest estymacja trajektorii robota przy równoczesnym tworzeniu reprezentacji nieznanego wcześniej otoczenia. W ujęciu probabilistycznym zarówno stan robota, jak i elementy mapy traktowane są jako zmienne losowe estymowane na podstawie kolejnych pomiarów i modelu ruchu [2].

Współczesne systemy SLAM można porównywać według kilku niezależnych kryteriów. Pierwszym z nich jest metoda estymacji stanu:

- **Metody oparte na filtracji** wykorzystują między innymi rozszerzone, iterowane lub bezśladowe filtry Kalmana. Pomiary są przetwarzane sekwencyjnie w etapach

predykcji i aktualizacji stanu. Podejście to umożliwia estymację wykonywaną z dużą częstotliwością, jednak jego dokładność zależy od poprawności modeli ruchu, pomiarów i szumów. Przykładem jest FAST-LIO2, który realizuje ścisłą integrację pomiarów LiDAR-u i IMU z wykorzystaniem iterowanego filtru Kalmana oraz rejestruje surowe punkty bezpośrednio względem mapy utrzymywanej w strukturze *ikd-Tree* [53]. FAST-LIO2 należy jednak traktować przede wszystkim jako system odometrii laserowo-inercyjnej i lokalnego mapowania, ponieważ jego podstawowa wersja nie zawiera mechanizmu globalnego domykania pętli.

- **Metody oparte na optymalizacji grafowej** przedstawiają problem estymacji w postaci grafu czynników. Zmiennymi grafu mogą być pozy robota, prędkości, błędy systematyczne IMU oraz parametry elementów mapy. Czynniki opisują natomiast zależności wynikające z pomiarów odometrycznych, inercyjnych, satelitarnych, dopasowania skanów lub detekcji zamknięcia pętli. Optymalizacja grafu umożliwia uwzględnienie pomiarów pochodzących z różnych chwil i korygowanie wcześniej wyznaczonej trajektorii. Mechanizm ten ogranicza skumulowany dryft i zwiększa globalną spójność mapy, szczególnie po wykryciu zamknięcia pętli.

Niezależnym kryterium klasyfikacji jest sposób integracji danych sensorycznych. W systemach luźno powiązanych (ang. *loosely coupled*) wyniki działania oddzielnych estymatorów, na przykład odometrii LiDAR-owej i odbiornika GNSS, są łączone na poziomie wyznaczonych pozycji lub prędkości. W systemach ściśle powiązanych (ang. *tightly coupled*) pomiary z różnych czujników są uwzględniane w ramach wspólnego problemu estymacji stanu. Pozwala to zachować większą część informacji zawartej w pomiarach, lecz zwiększa złożoność obliczeniową oraz wymagania dotyczące kalibracji i synchronizacji.

Przykładem systemu ściśle integrującego dane jest LIO-SAM. Wykorzystuje on preintegrację pomiarów IMU do kompensacji zniekształcenia chmury punktów i predykcji ruchu, natomiast więzy pochodzące z odometrii LiDAR-owej, IMU, GNSS i zamknięć pętli wprowadzane są do grafu czynników. Optymalizacja inkrementalna realizowana za pomocą iSAM2 umożliwia estymację trajektorii i budowanie mapy w czasie rzeczywistym [42].

Odrębny podział dotyczy architektury systemu. MRG-SLAM jest zdecentralizowanym systemem wielorobotowym rozwijającym rozwiązania znane z pakietu *hdl-graph-slam*. Każdy robot utrzymuje własny graf pozy, a wymiana informacji pomiędzy robotami umożliwia wykrywanie międzyrobotowych zamknięć pętli i łączenie lokalnych map [41].

MOLA (ang. *Modular Optimization framework for Localization and mApping*) reprezentuje podejście modułowe. Framework dzieli system lokalizacji i mapowania na wymienne komponenty odpowiedzialne między innymi za obsługę sensorów, generowanie obserwacji, utrzymywanie reprezentacji świata oraz optymalizację. Architektura umożliwia stosowanie różnych reprezentacji stanu i mapy oraz integrację heterogenicznych źró-

deł pomiarów. Przewidziano w niej również mechanizm czasowego usuwania z pamięci operacyjnej fragmentów mapy, które nie są aktualnie wykorzystywane, co ogranicza zapotrzebowanie na pamięć podczas pracy na rozległych obszarach [8, 7].

Metody wykorzystujące uczenie maszynowe nie stanowią rozłącznej kategorii względem filtracji i optymalizacji grafowej. Modele uczone mogą zastępować lub wspomagać wybrane elementy systemu, takie jak wyznaczanie korespondencji, estymacja głębi, ekstrakcja cech lub odrzucanie obserwacji dynamicznych. Przykładem jest DROID-SLAM, w którym sieć rekurencyjna iteracyjnie aktualizuje estymaty pozy kamery i głębi pikselowej, współpracując z różniczkowalną warstwą gęstej optymalizacji *bundle adjustment* [46]. Rozszerzenie mapy geometrycznej o etykiety obiektów i rodzajów powierzchni jest natomiast zadaniem semantycznego SLAM-u i wymaga dodatkowego modułu segmentacji lub klasyfikacji.

2.4 Metody nawigacji

Ocena przejezdności terenu (ang. *traversability analysis*) jest istotnym elementem nawigacji robotów terenowych. Jej celem jest określenie, czy dany fragment środowiska może zostać bezpiecznie pokonany przez konkretną platformę mobilną. Ocena może uwzględniać geometrię terenu, między innymi nachylenie, nierówność i wysokość przeszkód, a także właściwości powierzchni, ograniczenia kinematyczne oraz stateczność robota. Wynik analizy jest często odwzorowywany na mapie kosztów, na której wartości komórek reprezentują względny koszt lub ryzyko przejazdu przez dany obszar [50].

Szczególnym wyzwaniem w środowisku terenowym jest rozróżnienie przeszkód nieprzejezdnych, takich jak pnie drzew i głazy, od roślinności, która może ugiąć się pod naciskiem robota. Skaner laserowy rejestruje punkty należące zarówno do obiektów sztywnych, jak i do trawy lub zarośli, dlatego analiza oparta wyłącznie na geometrii może prowadzić do zaklasyfikowania przejezdnej roślinności jako przeszkody. Informacje geometryczne są z tego względu uzupełniane segmentacją semantyczną obrazów lub chmur punktów. Sama etykieta semantyczna nie określa jednak bezpośrednio właściwości mechanicznych podłoża, dlatego może być dodatkowo łączona z informacją o wcześniejszych przejazdach lub reakcjach układu jezdnego.

Do segmentacji semantycznej środowisk nieustrukturyzowanych wykorzystuje się zarówno konwolucyjne sieci neuronowe, jak i architektury oparte na transformatorach wizyjnych. Mechanizmy uwagi umożliwiają modelowanie zależności pomiędzy odległymi obszarami obrazu, dzięki czemu wybrane modele ViT osiągają wysoką jakość segmentacji terenu. Nie można jednak mówić o jednoznacznym odejściu od sieci CNN, ponieważ uzyskiwane wyniki zależą od zbioru danych, architektury i sposobu uczenia, a istotnymi kryteriami w robotyce pozostają również czas inferencji i zapotrzebowanie na pamięć [26].

Alternatywnym kierunkiem są metody uczące ocenę przejezdności na podstawie wcze-

śniejszych doświadczeń robota. De Miguel i współautorzy zaproponowali rozwiązanie wykorzystujące wielowarstwową mapę zawierającą informacje o wysokości i teksturze terenu. Dane zbierane podczas początkowego przejazdu sterowanego przez operatora są przekształcane przez wariacyjny autoenkoder do postaci wektorów cech, a następnie grupowane za pomocą algorytmu BIRCH. Podczas autonomicznej pracy podobieństwo nowych obserwacji do utworzonych klastrów służy do wyznaczania oceny przejezdności. Metoda ogranicza konieczność przygotowywania semantycznie etykietowanych danych dla każdego nowego środowiska i może aktualizować reprezentację przejezdnego terenu na podstawie kolejnych przejazdów [31].

2.4.1 Algorytmy planowania ruchu

Planowanie ruchu w terenie stanowi rozszerzenie nawigacji płaskiej o dodatkowe stopnie swobody i ograniczenia wynikające z ukształtowania podłoża (analiza 2.5D lub 3D). Jest to proces hierarchiczny, składający się z planera globalnego (wyznaczającego optymalną trasę na mapie o dużym zasięgu) oraz planera lokalnego (odpowiedzialnego za dynamiczne omijanie przeszkód i wyliczanie zadanych prędkości dla układu napędowego w pętli sterowania o wysokiej częstotliwości) [58].

W literaturze wyróżnia się cztery główne rodziny algorytmów planowania:

- **Metody przeszukiwania grafów** - bazujące na dyskretyzacji przestrzeni roboczej (np. siatki zajętości). Problem nawigacji sprowadza się tu do wyznaczenia najkrótszej ścieżki w grafie. Najpopularniejsze rozwiązania to algorytm Dijkstry [21], jego zoptymalizowana heurystycznie wersja A* [57] oraz wariant D* (Dynamic A*), przystosowany do środowisk zmiennych, umożliwiający wdyajną zmianę trasy bez konieczności przeliczania całego grafu [40].

- **Metody probabilistyczne** - dedykowane dla wielowymiarowych przestrzeni konfiguracyjnych. Zamiast budować pełną siatkę, wykorzystują losowe próbkowanie. Należą do nich algorytmy PRM (Probabilistic Roadmaps), skuteczne w środowiskach statycznych [48], oraz struktury drzewiaste RRT (Rapidly-exploring Random Trees) i ich asymptotycznie optymalny wariant RRT*, które charakteryzują się wysoką efektywnością w szybkim odnajdywaniu bezkolizyjnych ścieżek w złożonych przestrzeniach topologicznych [22, 32].

- **Pola potencjałowe (Artificial Potential Fields - APF)** - metoda reaktywna, w której robot zachowuje się jak cząstka w polu sił fizycznych. Punkt docelowy generuje siłę przyciągającą, a przeszkody siłę odpychającą. Jest to rozwiązanie wysoce wydajne obliczeniowo, idealne dla planerów lokalnych, jednak podatne na blokowanie się algorytmu w minimach lokalnych [39].

- **Algorytmy bioinspirowane** - optymalizacyjne metody heurystyczne stosowane przy wielokryterialnych funkcjach kosztu (np. minimalizacja czasu przejazdu z jednoczesnym unikaniem ostrych pochyłości). Obejmują one algorytmy genetyczne (GA) [17, 16] oraz metody optymalizacji mrówkowej (ACO), które poprzez naśladowanie mechanizmu feromonów skutecznie adaptują się do zmian w analizowanym terenie [49].

2.5 Nowoczesne architektury oprogramowania w robotyce

Wraz ze wzrostem złożoności algorytmów percepcji, planowania i sterowania, oprogramowanie robotów mobilnych wyewoluowało z architektur monolitycznych w stronę systemów rozproszonych. Współczesne systemy autonomiczne wymagają integracji wielu niezależnych procesów działających równolegle, co wymusiło zastosowanie specjalistycznego oprogramowania pośredniczącego.

Zdecydowanym standardem akademickim stał się system ROS (Robot Operating System). Jego pierwsza generacja (ROS1) znacząco przyspieszyła rozwój robotyki, jednak posiadała ograniczenia dyskwalifikujące ją z zastosowań krytycznych i terenowych [1]. Głównymi wadami ROS1 były:

- Scentralizowana architektura polegająca na jednym głównym brokerze (roscore) odpowiedzialnym za komunikację pomiędzy węzłami.
- Brak wbudowanego wsparcia dla architektur wykorzystujących wiele robotów.
- Brak wsparcia dla systemów czasu rzeczywistego.
- Brak wsparcia dla profili jakości usług (QoS - Quality of Service) - niezbędnego dla systemów krytycznych.

Odpowiedzią na te problemy jest ROS2, który całkowicie przebudował warstwę komunikacyjną, opierając ją na przemysłowym standardzie DDS (Data Distribution Service) [27]. Standard ten wprowadził architekturę w pełni rozproszoną, opartą na mechanizmie publikacji i subskrypcji bez centralnego punktu awarii. Kluczową innowacją dla robotyki terenowej, wprowadzoną w ROS 2, jest QoS. Pozwala to one na precyzyjne zarządzanie niezawodnością komunikacji, na przykład gwarantując dostarczenie krytycznych komend sterujących napędem, przy jednoczesnym akceptowaniu gubienia pojedynczych, przestarzałych wiadomości z chmurą punktów czujnika LIDAR w przypadku przeciążenia sieci pokładowej.

2.5.1 Paradygmat Drzew Zachowań

Wraz z ewolucją warstwy komunikacyjnej, zmianie uległ również sposób modelowania samej autonomii. Historycznie, zadania nawigacyjne opierano na maszynach stanów skończonych (FSM - Finite State Machines). Mimo intuicyjności dla prostych zadań, maszyny stanów stają się niezarządzalne przy złożonych zachowaniach wymagających reakcji na setki możliwych błędów i dynamicznych zmian w otoczeniu (zjawisko *state explosion*).

Nowoczesne frameworki nawigacyjne, takie jak stos Nav2 w środowisku ROS 2 [30], opierają się na Drzewach Zachowań (Behavior Trees - BT). Architektura ta, wywodząca się początkowo z branży gier komputerowych, charakteryzuje się hierarchiczną i w pełni modularną strukturą. Drzewa Zachowań pozwalają na łatwe tworzenie sekwencji zadań z wbudowanymi mechanizmami przywracania sprawności (recovery behaviors). Dzięki nim robot poruszający się w trudnym terenie może w sposób płynny i deterministyczny przerywać aktualnie wykonywane zadanie (np. jazdę do celu) na rzecz zachowania nadrzędnego (np. awaryjnego zatrzymania lub próby wyjechania z pułapki), a następnie powrócić do pierwotnego planu bez konieczności resetowania całego systemu.

Rozdział 3

Wstęp teoretyczny

3.1 Przestrzenny model kinematyczny

Tradycyjny model kinematyczny dla napędu różnicowego zakłada ruch platformy po płaskiej płaszczyźnie (2D). Jednakże dla robota terenowego operującego w nieustrukturyzowanym środowisku o zmiennym ukształtowaniu (wzniesienia, spadki, nierówności), konieczne jest rozszerzenie modelu do sześciu stopni swobody.

W przestrzeni trójwymiarowej stan robota opisany jest wektorem pozy q, zawierającym pozycję w globalnym układzie odniesienia $\{G\}$ oraz orientację przestrzenną wyrażoną za pomocą kątów Eulera:

$$q = [x, y, z, \phi, \theta, \psi]^T$$

gdzie:

- x, y, z - współrzędne środka układu lokalnego robota $\{R\}$ w układzie globalnym $\{G\}$,
- ϕ (roll) - kąt przechylenia (obrót wokół lokalnej osi X),
- θ (pitch) - kąt pochylenia (obrót wokół lokalnej osi Y),
- ψ (yaw) - kąt odchylenia (obrót wokół globalnej osi Z).

Wykorzystana w niniejszej pracy platforma [TODO: referencja rozdziału] charakteryzuje się napędem na cztery koła, dla którego zaimplementowano sterowanie burtowe (skid-steering). Z teoretycznego punktu widzenia, idealny model kinematyczny takiej platformy sprowadza się do modelu klasycznego napędu różnicowego (differential drive). W takim układzie zakłada się, że ruch platformy realizowany jest poprzez różnicowanie prędkości kół lewej i prawej strony pojazdu [11].

Układ kinematyczny sterowania burtowego generuje prędkości wyłącznie w płaszczyźnie lokalnej robota. Na podstawie obrotów kół, platforma wymusza wzdłużną prędkość

liniową v (wzdłuż osi X) oraz prędkość obrotową ω (wokół osi Z). W wyidealizowanym modelu z brakiem poślizgu poprzecznego, lokalny wektor prędkości robota wynosi $V_R = [v, 0, 0]^T$.

Aby wyznaczyć transformację pozycji robota w przestrzeni trójwymiarowej, należy rzutować prędkość lokalną na układ globalny przy użyciu macierzy obrotu $R_R^G(\phi, \theta, \psi)$:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = R_R^G(\phi, \theta, \psi) \begin{bmatrix} v \\ 0 \\ 0 \end{bmatrix}$$

Po wymnożeniu przez pełną macierz obrotu, różniczkowe równania ruchu pojazdu (przy założeniu zgodności z konwencją lotniczą) przyjmują postać:

$$\dot{x} = v \cos(\theta) \cos(\psi)$$

$$\dot{y} = v \cos(\theta) \sin(\psi)$$

$$\dot{z} = -v \sin(\theta)$$

Powyższe równania wyraźnie pokazują, że w środowisku terenowym wzdłużna prędkość v generowana przez koła napędowe rozkłada się na wszystkie trzy osie układu globalnego. Kąty przechyłu (ϕ) i pochylenia (θ) w tym modelu nie są stopniami swobody bezpośrednio sterowanymi przez napędy robota. Stanowią one ograniczenia wymuszane fizycznie przez geometrię podłoża, z którym stykają się koła platformy. Zmiana wysokości ($\dot{z}$) zależy wprost od kąta pochylenia robota θ w danym momencie.

Również pochodna kąta odchylenia (yaw rate w układzie globalnym) jest sprzężona z kątami przechyłu i pochylenia. W ogólnym przypadku terenowym, relacja między zadaną przez koła prędkością obrotową ω a globalną zmianą kątów Eulera wymaga przemnożenia przez macierz transformacji prędkości kątowych, co stanowi istotne utrudnienie dla algorytmów odometrii zliczanej (dead reckoning).

3.1.1 Ograniczenia modeli kinematycznych w środowisku terenowym

Przedstawiony idealny model kinematyczny stanowi fundament dla działania niskopoziomowych sterowników silników, pozwalając na zamianę żądanych prędkości robota na obroty poszczególnych kół. Należy jednak stanowczo podkreślić, że w środowisku terenowym, wykorzystanie tego modelu do estymacji pozycji robota jest obarczone krytycznymi błędami.

Współczesne badania nad platformami o sterowaniu burtowym dowodzą, że czysto analityczne modele kinematyczne ulegają szybkiej degradacji, szczególnie podczas wyko-

nywania manewrów skrętu przy wyższych prędkościach. Wynika to z faktu, że roboty tego typu z natury swojej mechaniki wymagają fizycznego poślizgu kół do zmiany orientacji. W nieustrukturyzowanym terenie o zmiennym współczynniku tarcia (np. mokra trawa, błoto, piasek), ignorowanie nieliniowych zjawisk na styku opony z podłożem prowadzi do drastycznego wzrostu błędów w estymacji kątowej i liniowej [47].

Dodatkowo, zjawisko to jest potęgowane przez przestrzenną naturę terenu. Pokonywanie wzniesień i nierówności wywołuje ruch w osiach pionowych, co przy zastosowaniu płaskiego modelu 2D skutkuje fałszywą interpretacją przebytej drogi.

Z powyższych względów, analityczny model odometrii kół jest całkowicie niewystarczający jako samodzielne źródło informacji o pozycji dla robotów terenowych. W najnowszych rozwiązaniach odchodzi się od prób stworzenia idealnego modelu matematycznego opony w terenie, na rzecz zaawansowanych algorytmów uczenia maszynowego (np. procesów gaussowskich) lub głębokiej fuzji sensorycznej. W niniejszej pracy przyjęto podejście, w którym błędy modelu kinematycznego są na bieżąco kompensowane przez systemy inercyjne oraz algorytmy dopasowywania skanów z czujników LIDAR.

3.2 Technologie sensoryczne w robotyce terenowej

Percepcja otoczenia stanowi krytyczny element systemów autonomicznych. W warunkach terenowych, gdzie tradycyjne modele kinematyczne zawodzą z powodu zmiennej interakcji kół z podłożem, niezawodna nawigacja wymaga zastosowania komplementarnych technologii sensorycznych. Czujniki te dzieli się na dwie główne grupy: ekstroceptywne (mierzące parametry środowiska zewnętrznego, np. skanery laserowe, kamery) oraz proprioceptywne (mierzące stan wewnętrzny robota, np. jednostki inercyjne) [28].

3.2.1 Skanery laserowe

Skanery laserowe LiDAR (ang. *Light Detection and Ranging*) są aktywnymi czujnikami optycznymi służącymi do bezpośredniego wyznaczania odległości do elementów otoczenia. Czujnik emituje promieniowanie laserowe, rejestruje sygnał powracający po odbiciu od powierzchni a następnie łączy zmierzoną odległość z kierunkiem emisji wiązki. Powtarzanie tego procesu dla kolejnych kierunków pozwala odtworzyć geometrię otoczenia w postaci skanu dwuwymiarowego albo trójwymiarowej chmury punktów. Ponieważ skaner jest źródłem wykorzystywanego promieniowania, jego działanie nie wymaga oświetlenia sceny światłem widzialnym. Nie oznacza to jednak całkowitej niezależności od warunków oświetleniowych, ponieważ silne promieniowanie słoneczne może zwiększać poziom sygnału tła docierającego do detektora [4].

Najczęściej stosowaną w robotyce mobilnej metodą pomiaru odległości jest bezpośredni pomiar czasu przelotu impulsu (ang. *direct Time of Flight*, dToF). Krótki impuls

laserowy zostaje wysłany w kierunku otoczenia, a układ pomiarowy wyznacza czas Δt pomiędzy jego emisją i rejestracją sygnału powrotnego. Odległość d od powierzchni odbijającej jest określona zależnością

$$d = \frac{c_m \Delta t}{2}, \tag{3.1}$$

gdzie c_m oznacza prędkość światła w ośrodku, w którym propaguje się wiązka. Współczynnik 1/2 wynika z uwzględnienia drogi impulsu od emitera do obiektu oraz z powrotem do detektora. Rozdzielczość czasowa układu pomiarowego bezpośrednio wpływa zatem na rozdzielczość wyznaczanej odległości. Oprócz metody dToF stosowane są również rozwiązania wykorzystujące pomiar przesunięcia fazowego fali modulowanej oraz technikę FMCW (ang. *Frequency-Modulated Continuous Wave*). W skanerach FMCW odległość jest wyznaczana na podstawie różnicy częstotliwości sygnału nadawanego i odbieranego, co umożliwia także pomiar prędkości radialnej obiektu. Rozwiązania impulsowe pozostają jednak powszechne w skanerach wykorzystywanych do mapowania i nawigacji robotów mobilnych [4, 38].

Skanery laserowe można klasyfikować według kilku niezależnych kryteriów. Ze względu na wymiar odwzorowywanej przestrzeni wyróżnia się:

- **skanery 2D** - kierujące wiązkę w obrębie jednej płaszczyzny i zwracające zbiór pomiarów $[d_i, \alpha_i]$, gdzie d_i jest odległością, a α_i kątem skanowania.
- **skanery 3D** - wykonujące pomiary dla wielu wartości azymutu (kątu skanowania) i elewacji, dzięki czemu bezpośrednio odwzorowują przestrzenną geometrię otoczenia.

Wymiar danych nie określa sposobu kierowania wiązką. Przykładowo, wielokanałowy skaner 3D może wykorzystywać zestaw emiterów ustawionych pod stałymi kątami elewacji oraz wspólny ruch obrotowy w osi pionowej. Ze względu na konstrukcję układu skanującego można natomiast wyróżnić następujące grupy urządzeń [38]:

- **skanery mechaniczne** - wykorzystujące obrót całej głowicy lub elementu układu optycznego. Konstrukcje tego typu zapewniają zazwyczaj szerokie pole widzenia i regularny wzór próbkowania, lecz zawierają części ruchome narażone na zużycie i oddziaływanie drgań.
- **skanery hybrydowe, określane także jako quasi-solid-state** - wykorzystujące miniaturowe elementy ruchome, na przykład zwierciadła MEMS. Umożliwiają zmniejszenie rozmiaru urządzenia, ale ich pole widzenia jest zwykle węższe niż w przypadku skanerów obrotowych, a rozkład punktów może zależeć od przebiegu sterującego zwierciadłem.
- **skanery półprzewodnikowe (solid-state)** - pozbawione mechanicznie poruszanych elementów układu skanującego. Do tej grupy należą między innymi czujniki

typu *flash*, oświetlające jednocześnie całe pole widzenia, oraz układy z optyczną anteną fazowaną (OPA), w których kierunek wiązki jest sterowany elektronicznie. Brak części ruchomych zwiększa odporność mechaniczną, lecz parametry zasięgu, rozdzielczości kątowej i pola widzenia zależą od zastosowanej architektury.

W przypadku skanera 3D pojedynczy pomiar można opisać odległością d_i, kątem skanowania α_i oraz kątem elewacji β_i. Współrzędne punktu w układzie związanym ze skanerem $\{S\}$ wyznaczane są wówczas następująco:

$$\mathbf{p}_i^S = \begin{bmatrix} d_i \cos\beta_i \cos\alpha_i \\ d_i \cos\beta_i \sin\alpha_i \\ d_i \sin\beta_i \end{bmatrix}. \tag{3.2}$$

Zbiór punktów $\mathcal{P} = \{\mathbf{p}_i^S\}_{i=1}^N$ tworzy chmurę punktów jednego skanu. Wiadomość pomiarowa może zawierać również intensywność sygnału powrotnego, numer kanału, znacznik czasu oraz informację o kolejności odbicia. Intensywność nie jest bezpośrednią miarą współczynnika odbicia powierzchni, ponieważ zależy między innymi od odległości, kąta padania wiązki, właściwości materiału i charakterystyki odbiornika. Przed wykorzystaniem danych przez algorytmy nawigacji punkty przekształca się z układu skanera do układu robota albo mapy:

$$\mathbf{p}_i^B = \mathbf{R}_S^B \mathbf{p}_i^S + \mathbf{t}_S^B, \tag{3.3}$$

gdzie $\mathbf{R}_S^B$ oraz $\mathbf{t}_S^B$ opisują parametry kalibracji zewnętrznej skanera względem układu bazowego robota $\{B\}$. Dokładność tej transformacji oraz synchronizacja czasowa czujników mają bezpośredni wpływ na zgodność kolejnych skanów.

Do podstawowych parametrów skanera należą: minimalny i maksymalny zasięg, dokładność oraz powtarzalność pomiaru odległości, poziome i pionowe pole widzenia, rozdzielczość kątowa, częstotliwość skanowania, liczba generowanych punktów w jednostce czasu oraz liczba rejestrowanych odbić. Parametry te należy analizować łącznie. Zwiększenie liczby punktów nie musi prowadzić do poprawy odwzorowania terenu, jeżeli pole widzenia skanera nie obejmuje podłoża bezpośrednio przed robotem albo rozkład kątów elewacji pozostawia pomiędzy warstwami znaczne odstępy.

Jakość danych jest zależna również od właściwości obserwowanej powierzchni. Materiały o małym współczynniku odbicia mogą ograniczać zasięg pomiaru, natomiast powierzchnie szklane, silnie połyskliwe lub obserwowane pod dużym kątem padania mogą prowadzić do braku pomiaru, odbić wielodrogowych albo wyznaczenia nieprawidłowej odległości. Mgła, deszcz, śnieg i pył powodują rozpraszanie oraz tłumienie wiązki, co zmniejsza zasięg użyteczny i może generować punkty nieodpowiadające trwałym elementom otoczenia [37].

Istotnym źródłem błędu na platformie mobilnej jest także zniekształcenie ruchowe

skanu. Punkty jednego obrotu nie są rejestrowane w tej samej chwili, a robot może w czasie ich akwizycji zmieniać pozycję i orientację. Traktowanie całej chmury jako pomiaru wykonanego dla jednej pozy robota prowadzi do zniekształcenia geometrii, szczególnie podczas szybkiego ruchu lub przejazdu po nierównościach. Kompensację tego zjawiska, określaną jako *deskewing*, realizuje się z wykorzystaniem znaczników czasu punktów i estymaty ruchu pochodzącej najczęściej z jednostki IMU, odometrii albo bieżącego wyniku estymatora stanu [43].

W systemach autonomicznych chmury punktów są wykorzystywane między innymi do dopasowywania kolejnych skanów, estymacji ruchu, budowy mapy, segmentacji podłoża oraz detekcji przeszkód. Skaner 3D dostarcza informacji metrycznej przydatnej w środowisku terenowym zarówno w dzień, jak i w nocy, jednak nie stanowi samodzielnie niezawodnego źródła pełnej informacji o stanie robota. Obszary o małej liczbie cech geometrycznych, zakłócenia atmosferyczne i zniekształcenia ruchowe uzasadniają integrację danych laserowych z pomiarami IMU, GNSS i systemów wizyjnych. Taka komplementarność czujników stanowi podstawę systemu percepcji opracowanego w niniejszej pracy.

3.2.2 Kamery głębi oparte na pomiarze czasu przelotu (ToF)

Uzupełnieniem skanerów laserowych są matrycowe kamery głębi ToF (ang. *Time of Flight*). Są to aktywne czujniki optyczne, które oświetlają obserwowaną scenę promieniowaniem podczerwonym, a następnie wyznaczają odległość oddzielnie dla każdego piksela matrycy. W przeciwieństwie do skanerów mechanicznych nie wymagają sekwencyjnego kierowania pojedynczej wiązki w kolejne punkty przestrzeni, dzięki czemu umożliwiają jednoczesne pozyskanie informacji o głębi w całym polu widzenia [23].

W kamerach tego typu najczęściej stosowany jest pośredni pomiar czasu przelotu (iToF, ang. *indirect Time of Flight*). Emitowane promieniowanie jest modulowane amplitudowo, a odległość wyznaczana jest na podstawie przesunięcia fazowego $\Delta\varphi$ pomiędzy sygnałem wyemitowanym i odebranym:

$$d = \frac{c}{4\pi f_m}\Delta\varphi, \tag{3.4}$$

gdzie c oznacza prędkość światła w ośrodku, a f_m jest częstotliwością modulacji [19]. Ze względu na okresowość pomiaru fazy jednoznaczny zakres pomiarowy jest ograniczony, dlatego w praktycznych rozwiązaniach mogą być wykorzystywane różne częstotliwości modulacji.

Wynikiem działania kamery jest mapa głębi, w której każdemu pikselowi przypisana jest zmierzona odległość. Czujnik może dostarczać również obraz amplitudy lub mapę jakości pomiaru, ułatwiającą odrzucanie punktów o małej wiarygodności [13]. Kamery ToF charakteryzują się gęstym odwzorowaniem przestrzeni i dużą częstotliwością odświeżania, dlatego znajdują zastosowanie w detekcji przeszkód w bliskim otoczeniu robota. Ich

ograniczeniami są stosunkowo niewielki zasięg oraz podatność na silne promieniowanie słoneczne, odbicia wielodrogowe i właściwości optyczne obserwowanych powierzchni.

3.2.3 Inercyjne jednostki pomiarowe (IMU)

Inercyjna jednostka pomiarowa (IMU, ang. *Inertial Measurement Unit*) jest czujnikiem proprioceptywnym dostarczającym informacji o ruchu platformy. Współczesne układy IMU realizowane są najczęściej w technologii mikroukładów elektromechanicznych MEMS (ang. *Microelectromechanical Systems*). Typowy układ o sześciu osiach pomiarowych, określany często jako IMU 6-DoF, składa się z trójosiowego akcelerometru oraz trójosiowego żyroskopu [45]. Niektóre moduły zawierają dodatkowo magnetometr, wykorzystywany do estymacji orientacji względem ziemskiego pola magnetycznego.

Żyroskop mierzy prędkość kątową, natomiast akcelerometr nie wyznacza bezpośrednio przyspieszenia liniowego obiektu w układzie globalnym. Jego pomiarem jest tak zwana siła właściwa, czyli różnica pomiędzy przyspieszeniem kinematycznym i przyspieszeniem grawitacyjnym, wyrażona w układzie współrzędnych czujnika. Model pomiarowy akcelerometru można zapisać jako

$$\tilde{\mathbf{a}}^I = \mathbf{R}_W^I \left(\mathbf{a}^W - \mathbf{g}^W \right) + \mathbf{b}_a + \mathbf{n}_a, \tag{3.5}$$

gdzie $\tilde{\mathbf{a}}^I$ jest wektorem zmierzonym przez akcelerometr w układzie IMU $\{I\}$, $\mathbf{a}^W$ oznacza rzeczywiste przyspieszenie liniowe w układzie globalnym $\{W\}$, $\mathbf{g}^W$ jest wektorem przyspieszenia grawitacyjnego, a $\mathbf{R}_W^I$ opisuje obrót z układu globalnego do układu czujnika. Wektory $\mathbf{b}_a$ oraz $\mathbf{n}_a$ reprezentują odpowiednio błąd systematyczny i szum pomiarowy.

Model pomiarowy żyroskopu ma postać

$$\tilde{\boldsymbol{\omega}}^I = \boldsymbol{\omega}^I + \mathbf{b}_g + \mathbf{n}_g, \tag{3.6}$$

gdzie $\tilde{\boldsymbol{\omega}}^I$ jest zmierzoną prędkością kątową, $\boldsymbol{\omega}^I$ jej wartością rzeczywistą wyrażoną w układzie IMU, natomiast $\mathbf{b}_g$ i $\mathbf{n}_g$ oznaczają błąd systematyczny oraz szum żyroskopu [14].

Błędy $\mathbf{b}_a$ i $\mathbf{b}_g$ nie są w praktyce stałe. Ich wartości zmieniają się między innymi pod wpływem temperatury i czasu pracy układu. W estymatorach stanu często modeluje się je jako procesy błądzenia losowego:

$$\dot{\mathbf{b}}_a = \mathbf{n}_{ba}, \qquad \dot{\mathbf{b}}_g = \mathbf{n}_{bg}, \tag{3.7}$$

gdzie $\mathbf{n}_{ba}$ i $\mathbf{n}_{bg}$ są szumami opisującymi zmianę błędów systematycznych. Bardziej rozbudowane modele mogą uwzględniać również błędy skali, nieprostopadłość osi pomiarowych oraz wpływ temperatury.

W przypadku nieruchomego czujnika żyroskop powinien wskazywać wartości bliskie zeru. Akcelerometr nie wskazuje jednak zerowego wektora, ponieważ rejestruje siłę wła-

ściwą związaną z oddziaływaniem grawitacji. Dla nieruchomego i wypoziomowanego czujnika wartość bezwzględna jego wektora pomiarowego powinna wynosić w przybliżeniu $9.81\,\mathrm{m/s^2}$. Właściwa interpretacja tego pomiaru wymaga znajomości orientacji czujnika względem układu globalnego.

Prędkość kątowa z żyroskopu jest całkowana w celu aktualizacji orientacji platformy, reprezentowanej najczęściej za pomocą kwaternionu lub macierzy obrotu. Po skompensowaniu błędu systematycznego i grawitacji pomiary akcelerometru mogą być wykorzystane do wyznaczenia prędkości oraz pozycji:

$$\dot{\mathbf{v}}^W = \mathbf{R}_I^W \left(\tilde{\mathbf{a}}^I - \mathbf{b}_a\right) + \mathbf{g}^W, \qquad \dot{\mathbf{p}}^W = \mathbf{v}^W. \tag{3.8}$$

Całkowanie powoduje narastanie błędów estymacji. Stały błąd akcelerometru prowadzi do liniowego wzrostu błędu prędkości i kwadratowego wzrostu błędu pozycji. Błąd żyroskopu powoduje natomiast nieprawidłową estymację orientacji, a w konsekwencji również błędne odjęcie składowej grawitacyjnej od pomiarów akcelerometru. Z tego względu samodzielna nawigacja inercyjna wykorzystująca czujniki klasy MEMS zachowuje dokładność jedynie w ograniczonym przedziale czasu.

Jednostki IMU generują pomiary zazwyczaj z częstotliwością wyższą niż skanery laserowe, dzięki czemu mogą być wykorzystywane do predykcji ruchu pomiędzy kolejnymi skanami. W algorytmach odometrii laserowo-inercyjnej dane inercyjne służą do propagacji stanu robota, natomiast dopasowanie chmur punktów ogranicza narastający dryft. Integracja może być realizowana za pomocą filtrów Kalmana lub optymalizacji grafu czynników [54, 43].

Dokładność fuzji danych zależy nie tylko od parametrów samych czujników, ale również od synchronizacji czasowej oraz kalibracji transformacji pomiędzy układami współrzędnych IMU i skanera. Nawet niewielkie przesunięcie czasowe lub błąd orientacji pomiędzy czujnikami może prowadzić do nieprawidłowej kompensacji ruchu i pogorszenia jakości dopasowania chmur punktów.

3.3 Teoria estymacji stanu i fuzji danych sensorycznych

Estymacja stanu polega na wyznaczaniu wielkości opisujących ruch robota na podstawie niepełnych i obarczonych błędami pomiarów. W przypadku pojazdu terenowego pomiar obrotów kół nie pozwala jednoznacznie odtworzyć przemieszczenia, a całkowanie sygnałów z czujnika IMU prowadzi do narastania błędu. Ponieważ obserwacje są obarczone szumem, wynik estymacji powinien być rozpatrywany wraz z jego niepewnością.

Fuzja danych sensorycznych umożliwia wykorzystanie uzupełniających się informacji pochodzących z różnych źródeł (czujników). Pomiary IMU pozwalają przewidywać zmiany

ruchu pomiędzy kolejnymi skanami, natomiast obserwacje laserowe dostarczają informacji o geometrii otoczenia ograniczających dryf. Integrację tych informacji można realizować za pomocą filtracji, jak w metodzie FAST-LIO, lub optymalizacji grafu czynników, jak w LIO-SAM [55, 42].

Niepewność wyniku estymacji opisuje się między innymi za pomocą macierzy kowariancji. Dla wektora błędu $\mathbf{e}_k$ o zerowej wartości oczekiwanej ma ona postać

$$\mathbf{P}_k = \mathbb{E}\left[\mathbf{e}_k \mathbf{e}_k^{\mathsf{T}}\right]. \tag{3.9}$$

Elementy diagonalne określają wariancje poszczególnych składowych błędu, a pozostałe elementy opisują ich wzajemne zależności. Kowariancja umożliwia więc ilościowe określenie niepewności estymaty oraz uwzględnienie jej podczas łączenia informacji z różnych źródeł [3].

Rozszerzony filtr Kalmana (EKF, ang. *Extended Kalman Filter*) służy do estymacji stanu układów opisanych modelami nieliniowymi. Jego działanie obejmuje dwa etapy: predykcję i korekcję. W predykcji model ruchu pozwala przewidzieć kolejny stan oraz jego niepewność. Po otrzymaniu pomiaru estymata jest korygowana na podstawie różnicy pomiędzy obserwacją a wartością przewidywaną przez model pomiarowy. Podstawowy schemat tych operacji, dla wektorowego opisu stanu, można zapisać jako

$$\widehat{\mathbf{x}}_k^- = f\left(\widehat{\mathbf{x}}_{k-1}^+, \mathbf{u}_k\right), \tag{3.10}$$

$$\widehat{\mathbf{x}}_k^+ = \widehat{\mathbf{x}}_k^- + \mathbf{K}_k \left[\mathbf{z}_k - h\left(\widehat{\mathbf{x}}_k^-\right)\right], \tag{3.11}$$

gdzie f jest modelem ruchu, h modelem pomiarowym, $\mathbf{u}_k$ wejściem modelu ruchu, a $\mathbf{z}_k$ obserwacją. Indeksy $-$ i $+$ oznaczają estymatę przed i po korekcji. Macierz wzmocnienia $\mathbf{K}_k$ określa wpływ obserwacji na aktualizację stanu. Jej wyznaczenie uwzględnia niepewność predykcji i pomiaru, natomiast nieliniowe modele są lokalnie linearyzowane [52].

W wariancie iteracyjnym filtru model pomiarowy jest ponownie linearyzowany wokół kolejnych przybliżeń stanu. Ma to na celu ograniczenie wpływu błędu linearyzacji podczas aktualizacji. Kolejne iteracje wykorzystują tę samą obserwację, a nie nowe, niezależne pomiary. Podejście to jest stosowane w metodach FAST-LIO i FAST-LIO2 do integracji danych laserowych i inercyjnych [55, 54].

Estymację można również sformułować jako problem optymalizacji, w którym poszukiwane stany minimalizują błędy zgodności z dostępnymi pomiarami. W grafie czynników obserwacje tworzą zależności pomiędzy wybranymi zmiennymi, na przykład stanami robota z różnych chwil. Mogą one wynikać z odometrii, pomiarów IMU albo dodatkowych obserwacji pozycji. Taką reprezentację wykorzystuje LIO-SAM [42]. Lokalna optymalizacja występuje także przy rejestracji chmur punktów, stanowiącej podstawę odometrii laserowej MOLA [7].

3.4 Algorytmy SLAM

Jednoczesna lokalizacja i mapowanie (SLAM, ang. *Simultaneous Localization and Mapping*) polega na wyznaczaniu pozy robota przy równoczesnym tworzeniu reprezentacji nieznanego otoczenia. Zadania te są ze sobą powiązane:

- Położenie obserwowanych elementów mapy zależy od oszacowanej pozycji i orientacji robota.
- Utworzona mapa może być następnie wykorzystywana do lokalizacji na podstawie kolejnych obserwacji [8].

W przypadku skanera laserowego stan geometryczny w chwili k można opisać macierzą transformacji jednorodnej

$$\mathbf{T}_k = \begin{bmatrix} \mathbf{R}_k & \mathbf{t}_k \\ \mathbf{0}^T & 1 \end{bmatrix} \in SE(3), \tag{3.12}$$

gdzie $\mathbf{R}_k \in SO(3)$ jest macierzą obrotu, a $\mathbf{t}_k \in \mathbb{R}^3$ wektorem położenia skanera względem układu mapy. Zbiór $SE(3)$ opisuje sztywne przekształcenia przestrzenne. Znajomość transformacji pomiędzy skanerem a punktem nawigacyjnym robota umożliwia następnie wyznaczenie pozy platformy [3].

3.4.1 Rejestracja chmur punktów

Podstawowym zadaniem lokalizacji laserowej jest wyznaczenie transformacji dopasowującej bieżącą chmurę punktów do danych odniesienia. W podejściu *scan-to-scan* porównuje się kolejne skany, natomiast w podejściu *scan-to-map* bieżący pomiar dopasowuje się do mapy zbudowanej z wcześniejszych obserwacji. Mapa lokalna pozwala wykorzystać informacje z większego fragmentu otoczenia, jednak jej jakość również zależy od dokładności wcześniej wyznaczonych póz [7].

Klasyczną metodą rejestracji jest algorytm ICP (ang. *Iterative Closest Point*). W kolejnych iteracjach wyznacza on korespondencje pomiędzy punktami obu zbiorów, a następnie oblicza obrót i przesunięcie zmniejszające błąd dopasowania. Dla ustalonych korespondencji wariant punkt-punkt minimalizuje funkcję

$$E_{\mathrm{pp}}(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^{N} \left\| \mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{c(i)} \right\|^2, \tag{3.13}$$

gdzie $\mathbf{p}_i$ jest punktem chmury wejściowej, $\mathbf{q}_{c(i)}$ odpowiadającym mu punktem zbioru odniesienia, a N liczbą zaakceptowanych par. Macierz $\mathbf{R}$ i wektor $\mathbf{t}$ przekształcają punkty wejściowe do układu odniesienia. Po aktualizacji transformacji korespondencje są wyznaczane ponownie. Obliczenia kończą się po osiągnięciu kryterium zbieżności albo limitu iteracji [5].

Inną funkcję błędu wykorzystuje dopasowanie punkt-płaszczyzna. Punkt wejściowy jest odnoszony do lokalnej powierzchni opisanej punktem $\mathbf{q}_{c(i)}$ i jednostkowym wektorem normalnym $\mathbf{n}_{c(i)}$:

$$E_{\mathrm{p}\pi}(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^{N} \left[\mathbf{n}_{c(i)}^{T} \left(\mathbf{R}\mathbf{p}_i + \mathbf{t} - \mathbf{q}_{c(i)} \right) \right]^2 . \tag{3.14}$$

Minimalizowana jest wówczas składowa błędu prostopadła do powierzchni. Takie więzy geometryczne wykorzystuje między innymi algorytm FAST-LIO2, integrując je z pomiarami IMU w iterowanym filtrze Kalmana [54].

Rejestracja wymaga odpowiedniego przybliżenia początkowego. Przy zbyt dużym błędzie pozy lub niewielkiej wspólnej części skanów algorytm może wyznaczyć nieprawidłowe dopasowanie. Przybliżenie ruchu może pochodzić z modelu kinematycznego, wcześniejszych wyników odometrii lub pomiarów inercyjnych. W LIO-SAM preintegracja IMU dostarcza zarówno przewidywania ruchu, jak i informacji potrzebnych do kompensacji zniekształcenia skanu [42].

Na wynik wpływa również przygotowanie chmury wejściowej. Ograniczenie zakresu pomiarów i zmniejszenie gęstości próbkowania pozwalają ograniczyć koszt obliczeń oraz udział punktów nieprzydatnych do estymacji. Nieprawidłowe korespondencje można odrzucać na podstawie odległości, a wpływ pozostałych obserwacji odstających ograniczać przez zastosowanie odpornych funkcji błędu [7].

3.4.2 Organizacja estymacji i optymalizacja trajektorii

W systemach SLAM często wyróżnia się warstwę przetwarzania obserwacji i warstwę estymacji. Warstwa przetwarzania odpowiada między innymi za przygotowanie danych, wyszukiwanie korespondencji i rozpoznawanie wcześniej obserwowanych miejsc. Druga wykorzystuje utworzone zależności pomiarowe do wyznaczania stanu robota i mapy. Jest to podział funkcjonalny, którego szczegółowa realizacja zależy od architektury systemu [9].

W rozwiązaniach opartych na optymalizacji grafowej kolejne pozy mogą tworzyć węzły grafu, a oszacowane przemieszczenia względne stanowią łączące je więzy. Pomiary są obarczone niepewnością, dlatego nie wszystkie zależności muszą być równocześnie spełnione. Optymalizacja wyznacza taki zbiór póz, który ogranicza łączny błąd zgodności z pomiarami, uwzględniając ich wiarygodność. Ustalenie wybranej pozy odniesienia usuwa możliwość dowolnego przesunięcia i obrotu całego grafu [3].

Nie każdy skan musi być przechowywany jako osobny węzeł. Wykorzystanie klatek kluczowych, wybieranych na przykład na podstawie przemieszczenia lub zmiany orientacji, ogranicza rozmiar problemu. W systemach laserowo-inercyjnych estymowane zmienne mogą dodatkowo obejmować prędkość i błędy systematyczne IMU, a graf czynników uwzględniać pomiary pochodzące z różnych źródeł [42].

3.4.3 Domykanie pętli i reprezentacja mapy

Sekwencyjna estymacja przemieszczeń prowadzi do narastania błędów, określanego jako dryf. Gdy robot powraca do wcześniej obserwowanego obszaru, możliwe jest wykrycie zamknięcia pętli (ang. loop closure). Dopasowanie aktualnych danych do wcześniejszych obserwacji dostarcza dodatkowego więzu pomiędzy odległymi w czasie stanami. W systemie grafowym ponowna optymalizacja może wówczas skorygować wcześniejszy przebieg trajektorii [3].

Rozpoznanie podobnego miejsca wymaga weryfikacji geometrycznej. Powtarzalne układy obiektów mogą prowadzić do błędnego powiązania różnych fragmentów środowiska. Nieprawidłowy więz zamknięcia pętli może zdeformować trajektorię i mapę, dlatego jego akceptacja powinna uwzględniać zgodność dopasowania oraz pozostałych informacji pomiarowych [9].

Sposób przechowywania mapy wpływa na możliwość jej późniejszej korekcji. Zachowanie klatek kluczowych wraz z obserwacjami i pozami umożliwia ponowne zbudowanie mapy metrycznej po optymalizacji trajektorii. Jest to bardziej elastyczne niż zapis wyłącznie jednej połączonej chmury, w której nie zachowano powiązań punktów z pomiarami źródłowymi. Takie rozdzielenie reprezentacji obserwacji od wynikowej mapy jest istotnym elementem podejścia metody MOLA [7].

Chmura punktów opisuje zarejestrowane powierzchnie, ale sama nie rozróżnia jawnie przestrzeni wolnej od niezaobserwowanej. Możliwe jest wykorzystanie reprezentacji objętościowej, w której przestrzeń dzieli się na komórki trójwymiarowe, czyli woksele. Przykładem jest OctoMap, wykorzystujący strukturę drzewa ósemkowego i probabilistyczną aktualizację zajętości. Informację o przestrzeni wolnej uzyskuje się przez analizę przebiegu promieni pomiarowych pomiędzy czujnikiem a punktami odbicia. Rozróżnienie obszarów zajętych, wolnych i nieznanych ma znaczenie przy planowaniu ruchu [20].

3.4.4 Ograniczenia w środowisku terenowym

Dokładność lokalizacji zależy od geometrii otoczenia. Wynika to również z postaci funkcji błędu punkt-płaszczyzna: dla pojedynczej idealnej płaszczyzny przesunięcie równoległe do niej nie zmienia odległości prostopadłych. Duża liczba punktów należących wyłącznie do płaskiego podłoża nie zapewnia jednakowej informacji o wszystkich składowych ruchu. Obecność powierzchni o różnych orientacjach dostarcza dodatkowych ograniczeń [54].

W terenie znaczenie mają także szybkie zmiany orientacji i drgania pojazdu. Pomiary inercyjne wspomagają estymację pomiędzy skanami, jednak ich wykorzystanie wymaga poprawnej kalibracji i synchronizacji z czujnikiem LIDAR. Obserwacje ruchomych obiektów oraz przemieszczającej się roślinności mogą natomiast naruszać założenie zgodności kolejnych pomiarów z nieruchomą sceną [42, 9].

Oceniając wyniki, należy zatem rozróżniać dokładność bieżącej lokalizacji, spójność mapy i skuteczność korekcji globalnej. Zapis chmury punktów nie oznacza sam w sobie, że system wykonuje domykanie pętli. W porównaniu metod istotne jest określenie, które z tych mechanizmów były rzeczywiście aktywne w badanej konfiguracji.

3.5 Modele planowania ruchu i omijania przeszkód

Planowanie ruchu polega na wyznaczaniu sposobu przemieszczania się robota od stanu początkowego do celu z uwzględnieniem otoczenia i ograniczeń kinematycznych, należy przy tym odróżnić ścieżkę, opisującą geometryczny przebieg ruchu, od trajektorii, która określa również jego przebieg w czasie. Wyznaczenie bezkolizyjnej ścieżki nie rozstrzyga więc, z jaką prędkością robot powinien pokonywać jej poszczególne fragmenty [25].

W typowej architekturze nawigacyjnej rozróżnia się planowanie globalne i sterowanie lokalne. Planer wyznacza ścieżkę do celu, natomiast kontroler przelicza ją na polecenia ruchu na podstawie bieżącej pozy i obserwacji otoczenia. Oba moduły korzystają z reprezentacji przeszkód i dopuszczalnych obszarów przejezdności [33].

3.5.1 Reprezentacja terenu i mapy kosztów

Mapa geometryczna uzyskana przez system SLAM wymaga przetworzenia do postaci użytecznej dla planera. Mapa kosztów przypisuje obszarom otoczenia wartości opisujące względną trudność przejazdu oraz wyróżnia obszary niedopuszczalne i nieznane. W stosie Navigation2 standardowa reprezentacja kosztów jest dwuwymiarowa. Warstwa wokselowa może przetwarzać dane przestrzenne z LiDAR-u lub kamery głębi, lecz jej wynik jest rzutowany do siatki używanej do planowania i sterowania [33, 35].

W nawigacji terenowej istotne jest również ukształtowanie powierzchni ziemii. Przykładem reprezentacji pośredniej jest mapa wysokościowa 2.5D, przypisująca komórkom płaszczyzny wysokość podłoża. Na podstawie lokalnej geometrii można oceniać nachylenie, nierówności i różnice wysokości. Ocena przejezdności zależy przy tym od właściwości platformy, między innymi prześwitu, wymiarów układu jezdnego i stateczności. Ten sam fragment terenu może być dopuszczalny dla jednego robota i nieprzejezdny dla innego [50].

Przetwarzanie chmury powinno zatem odróżniać podłoże od przeszkód oraz uwzględniać obszary, których geometrii nie rozpoznano dostatecznie. Sam brak punktów nie oznacza potwierdzonej przejezdności. Rozdzielczość siatki stanowi kompromis: drobne komórki umożliwiają dokładniejsze odwzorowanie przeszkód, ale zwiększają koszt obliczeń i zapotrzebowanie na pamięć [33].

Sprawdzanie kolizji powinno uwzględniać obrys platformy i jej orientację. Dodatkowo stosuje się inflację przeszkód, która nadaje podwyższone koszty komórkom w ich otoczeniu. Taki rozkład kosztów sprzyja prowadzeniu ścieżki z większym odstępem od przeszkód.

Inflacja i ocena kolizji pełnią jednak różne funkcje: preferowany odstęp od obiektu oraz geometryczna dopuszczalność danej pozy wymagają odpowiedniego uwzględnienia w planerze i kontrolerze [33, 34].

3.5.2 Planowanie globalne i kryterium kosztu

Problem planowania można przedstawić jako przeszukiwanie grafu. Węzły odpowiadają stanom robota, a krawędzie dopuszczalnym przejściom pomiędzy nimi. Zależnie od modelu stan może obejmować samo położenie albo również orientację. Wybrane przejścia powinny odpowiadać kinematyce pojazdu, ponieważ możliwość osiągnięcia pozycji docelowej nie oznacza, że każdy geometryczny sposób dotarcia do niej jest wykonalny [25].

Kryterium wyboru ścieżki może łączyć jej długość z trudnością przejazdu. Przykładowy model ma postać

$$J(P) = \sum_{i=1}^{m} \ell_i \left(1 + \lambda \overline{C}_i\right), \tag{3.15}$$

gdzie P jest ścieżką złożoną z m odcinków, ℓ_i długością odcinka, $\overline{C}_i \in [0, 1]$ znormalizowanym średnim kosztem obszarów, przez które on przebiega, a $\lambda \geq 0$ wagą trudności przejazdu. Odcinki prowadzące przez obszary niedopuszczalne są wykluczane. W tym modelu krótsza ścieżka może mieć większy koszt niż dłuższy przejazd przez łatwiejszy teren. Wykorzystanie kosztów zależnych od właściwości terenu jest stosowane w planowaniu dla pojazdów terenowych [50].

Przykładem algorytmu przeszukiwania jest A*, w którym priorytet rozwijania węzła n określa funkcja

$$f(n) = g(n) + h(n), \tag{3.16}$$

gdzie $g(n)$ oznacza najmniejszy dotychczas znaleziony koszt dotarcia do węzła, a $h(n)$ heurystyczne oszacowanie kosztu pozostałej drogi. Heurystyka dopuszczalna nie przeszacowuje tego kosztu. Jej postać powinna odpowiadać funkcji kosztu używanej podczas przeszukiwania grafu [18].

Wynik optymalizacji odnosi się do wybranego grafu i funkcji kosztu. Zmiana rozdzielczości, dopuszczalnych kierunków przejścia lub uwzględnianych składowych stanu może prowadzić do innej ścieżki. Dyskretny wynik może wymagać wygładzenia, przy czym modyfikacja jego geometrii wymaga ponownego sprawdzenia kolizji [25].

3.5.3 Śledzenie ścieżki i sterowanie lokalne

Kontroler lokalny wyznacza polecenia ruchu z uwzględnieniem błędu względem ścieżki oraz ograniczeń prędkości i przyspieszeń pojazdu. Dla robota o napędzie różnicowym lub sterowaniu burtowym polecenia te mogą być opisane prędkością wzdłużną v i kątową ω. Dobór dopuszczalnego sterowania powinien uwzględniać również możliwość zatrzymania

przed przeszkodą [15].

Metoda Pure Pursuit polega na wyborze punktu wyprzedzenia na ścieżce i wyznaczeniu krzywizny łuku prowadzącego do niego. Dla współrzędnych (x_L, y_L) tego punktu w układzie robota, przy osi x skierowanej do przodu i osi y w lewo (układ prawoskrętny), zależności mają postać

$$\kappa = \frac{2y_L}{L^2}, \qquad L = \sqrt{x_L^2 + y_L^2} > 0, \qquad \omega = v\kappa, \tag{3.17}$$

gdzie κ jest krzywizną zadanego ruchu, a L odległością od robota do punktu wyprzedzenia. Jej dobór wpływa na szybkość zbiegania do ścieżki i skłonność do oscylacji. Możliwe jest również uzależnienie odległości wyprzedzenia od prędkości [29].

Regulated Pure Pursuit (RPP) rozszerza tę zasadę o regulację prędkości, między innymi podczas ostrych skrętów i w pobliżu przeszkód. Przewidywany ruch jest sprawdzany pod kątem kolizji w ograniczonym horyzoncie. Wykrycie zagrożenia może prowadzić do zatrzymania pojazdu. RPP pozostaje jednak kontrolerem śledzącym zadaną ścieżkę i wyznaczenie alternatywnego przebiegu trasy wymaga współpracy z planerem [29].

Inne podejście reprezentuje metoda dynamicznego okna (DWA, ang. Dynamic Window Approach). W przestrzeni par (v, ω) analizowane są ruchy osiągalne przy ograniczonych przyspieszeniach i pozwalające na zatrzymanie przed przeszkodą. Kandydaci są oceniani na podstawie kierunku ruchu względem celu, odległości od przeszkód i prędkości. Pozwala to powiązać dobór sterowania z lokalną geometrią otoczenia [15].

3.5.4 Reakcja na przeszkody i ograniczenia terenowe

Nowa przeszkoda może zablokować ścieżkę wyznaczoną wcześniej. Jeżeli kontroler nie znajduje dopuszczalnego ruchu, konieczne może być zatrzymanie i ponowne planowanie. W szczególności wykrywanie kolizji przez RPP nie wyznacza samo w sobie alternatywnej trasy [29].

Znaczenie opóźnień można pokazać na uproszczonym modelu ruchu prostoliniowego. Jeżeli robot przez czas reakcji τ porusza się ze stałą prędkością $v \geq 0$, a następnie hamuje ze stałym opóźnieniem $a_b > 0$, droga do zatrzymania wynosi

$$d_{\text{stop}} = v\tau + \frac{v^2}{2a_b}. \tag{3.18}$$

Pierwszy składnik opisuje drogę przebytą przed rozpoczęciem hamowania, a drugi samo hamowanie. Model pokazuje, dlaczego dobór prędkości wymaga uwzględnienia zarówno aktualności danych, jak i zdolności zatrzymania platformy. Zależność dopuszczalnej prędkości od możliwości hamowania jest jednym z założeń DWA [15].

W terenie możliwości te zależą dodatkowo od nachylenia i właściwości podłoża. Sama mapa geometryczna nie dostarcza pełnej informacji o przyczepności ani nośności po-

wierzchni. Z tego względu analiza wyników nawigacji powinna obejmować nie tylko długość wyznaczonej ścieżki, lecz także skuteczność osiągania celu, przebieg ruchu oraz przyczyny zatrzymań. Rozdzielenie tych zagadnień ułatwia ocenę wpływu percepcji, lokalizacji i planowania na działanie całego systemu.

Rozdział 4

Architektura systemu i platformy sprzętowe

4.1 Autonomiczny robot terenowy Rebob

4.2 Architektura oprogramowania

4.3 Standardy tworzenia oprogramowania i przygotowanie systemu do wdrożeń

Rozdział 5

Środowisko symulacyjne

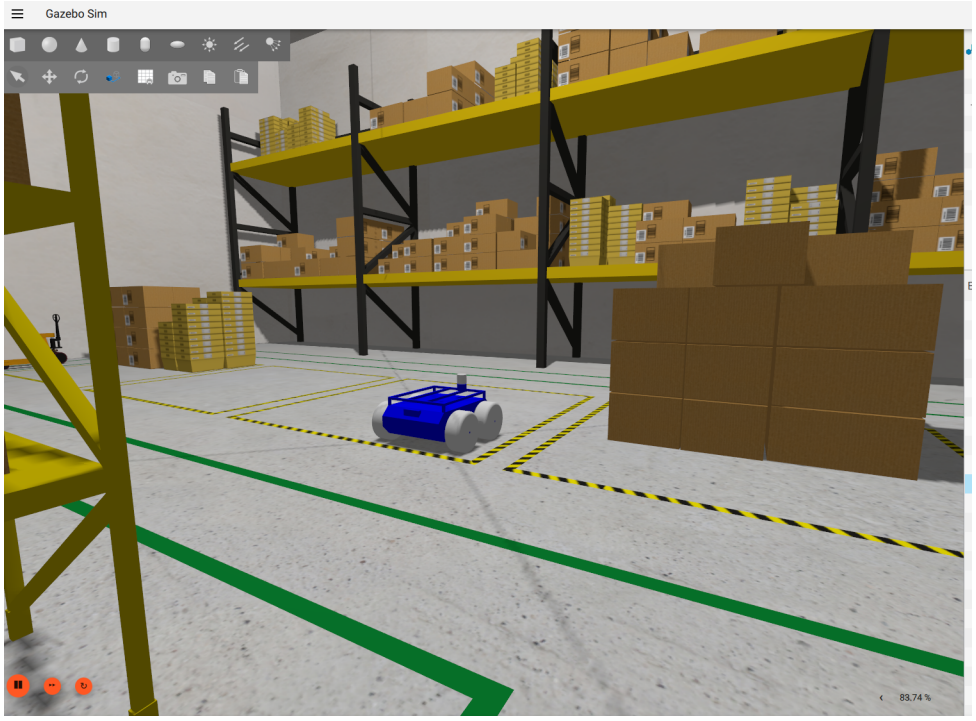
5.1 Modelowanie środowiska symulacyjnego w symulatorze Gazebo

Aby umożliwić płynne przeniesienie opracowanego systemu percepcji i nawigacji ze środowiska wirtualnego na rzeczywistą platformę mobilną, zdecydowano się na spójny, otwartoźródłowy stos technologiczny projektowany przez Open Source Robotics Foundation (OSRF). Rolę środowiska uruchomieniowego dla algorytmów autonomii pełni system ROS2 (Jazzy), natomiast do symulowania warunków terenowych i fizyki sensorów wykorzystano środowisko Gazebo (Harmonic).

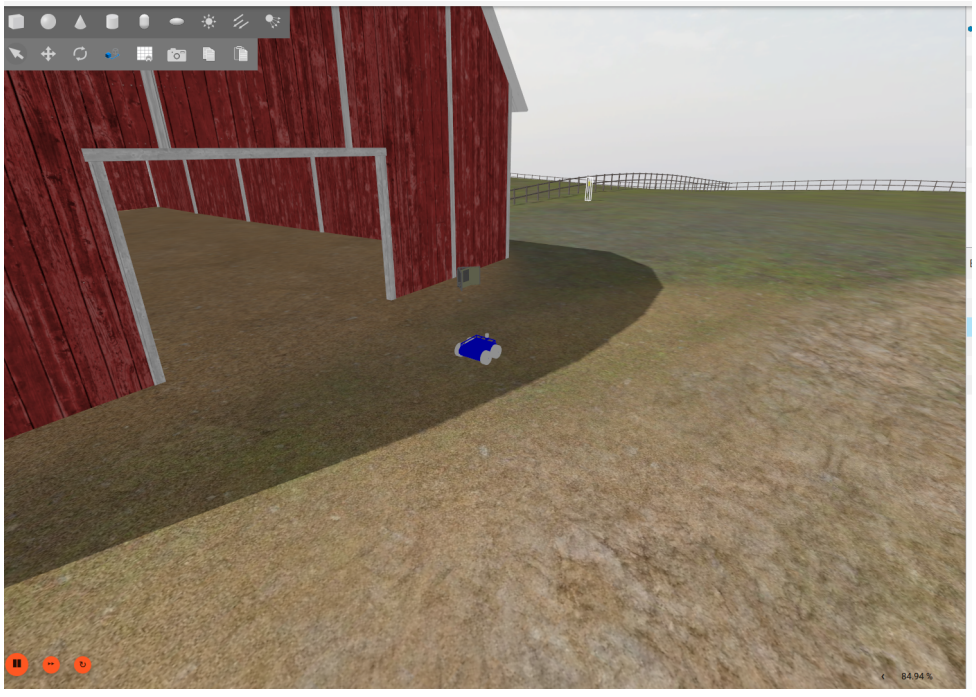
W celu najlepszego odzwierciedlenia rzeczywistego środowiska terenowego, w którym docelowo ma operować robot, wykorzystano mapy otoczenia udostępnione na otwartoźródłowej licencji. Podczas wyszukiwania odpowiedniego środowiska 3D kierowano się następującymi kryteriami:

- **Obecność przewyższeń terenu:** Niezbędny element środowiska terenowego. Bez zróżnicowanej rzeźby terenu niemożliwe jest kompleksowe testowanie paczek nawigacji i mapowania w przestrzeni trójwymiarowej
- **Obecność przeszkód:** Elementy takie jak drzewa, budynki, ławki czy krawężniki są niezbędne do weryfikacji algorytmów omijania przeszkód
- **Wielkość środowiska:** Duży obszar pozwala na przetestowanie wydajności algorytmów tworzenia map o znacznych rozmiarach oraz globalnej lokalizacji robota

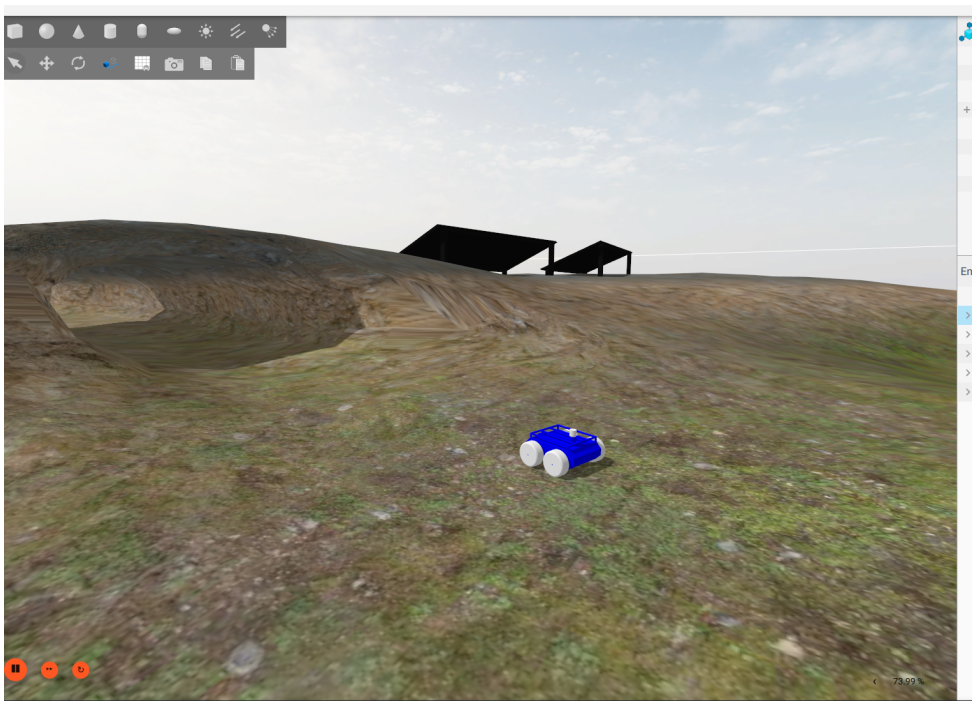
Posłużono się mapami dostępnymi w repozytorium clearpath_simulator firmy Clearpath Robotics. W szczególności mapy `pipeline.sdf` oraz `solar_farm.sdf` spełniły wymagania postawione względem ukształtowania terenu. Dodatkowo wykorzystano mapy industrial_warehouse.sdf oraz `warehouse.sdf` w celu przeprowadzenia testów porównawczych działania algorytmów nawigacji w terenie płaskim.



Rysunek 5.1: Widok symulowanego robota w środowisku industrial warehouse.



Rysunek 5.2: Widok symulowanego robota w środowisku solar farm.



Rysunek 5.3: Widok symulowanego robota w środowisku pipeline.

5.2 Tworzenie modeli kinematycznych i dynamicznych

W celu zamodelowania platformy mobilnej Rebob wykorzystano format URDF (Unified Robot Description Format). Proces tworzenia modelu rozpoczął się w środowisku CAD Autodesk Fusion, gdzie stworzono uproszczoną reprezentację geometrii robota. Tak przygotowany model wyeksportowano do formatu STL obsługiwanego przez silnik graficzny symulatora Gazebo.

Model URDF robota składa się z sześciu głównych członów (brył):

- 1x Korpus główny (`base_link`)
- 4x Koło napędowe
- 1x Skaner laserowy (LIDAR 3D)

W modelu zdefiniowano relacje pomiędzy każdą bryłą używając przekształceń statycznych. W formacie URDF każda relacja traktowana jest jako przegub. Warto zaznaczyć, że członem nie musi być rzeczywista bryła, ale może nim być również symulowana pozycja czujnika (np. IMU lub kamery 3D). W modelu wykorzystano następujące typy przegubów:

- **Obrotowe (continuous)** - zastosowane dla każdego koła platformy
- **Stałe (fixed)** - zastosowane dla elementów na sztywno połączonych z ramą (IMU, kamera 3D, LIDAR 3D, antena GPS)

Struktura modelu oparta jest na plikach `xacro`, które pozwalają na definicję struktury robota w formacie XML i parametryzację przekształceń. Format ten umożliwia modularyzację poprzez dzielenie definicji na wiele plików, a następnie zawieranie ich jako podmodułów w pliku głównym.

Zarządzanie napędem i kinematyką w środowisku symulacyjnym zrealizowano za pomocą frameworka `ros2_control`. Integracja symulatora Gazebo z `ros2_control` pozwala na płynne przenoszenie algorytmów sterowania z symulacji na rzeczywistą platformę sprzętową bez modyfikacji kodu wysokopoziomowego - zmianie ulega jedynie interfejs sprzętowy (hardware interface). W ramach przeprowadzonych prac wykorzystano standardową wtyczkę kinematyki dla napędu różnicowego (differential drive) i przekierowano sygnały sterujące na wirtualne silniki symulatora, przygotowując jednocześnie architekturę pod autorską wtyczkę sterownika silników BLDC typu VESC używaną w rzeczywistej implementacji.

```
<joint name="wheel_rl_joint" type="continuous">
  <origin xyz="-0.20582 0.439276 0.072471" rpy="0 0 -1.57"/>
  <parent link="base_link"/>
  <child link="wheel_rl"/>
  <axis xyz="-1.0 0.0 0.0"/>
</joint>

<joint name="velodyne_joint" type="fixed">
  <origin xyz="-0.23182 0.089629 0.366" rpy="0 0 0"/>
  <parent link="base_link"/>
  <child link="velodyne"/>
  <axis xyz="0.0 0.0 1.0"/>
</joint>
```

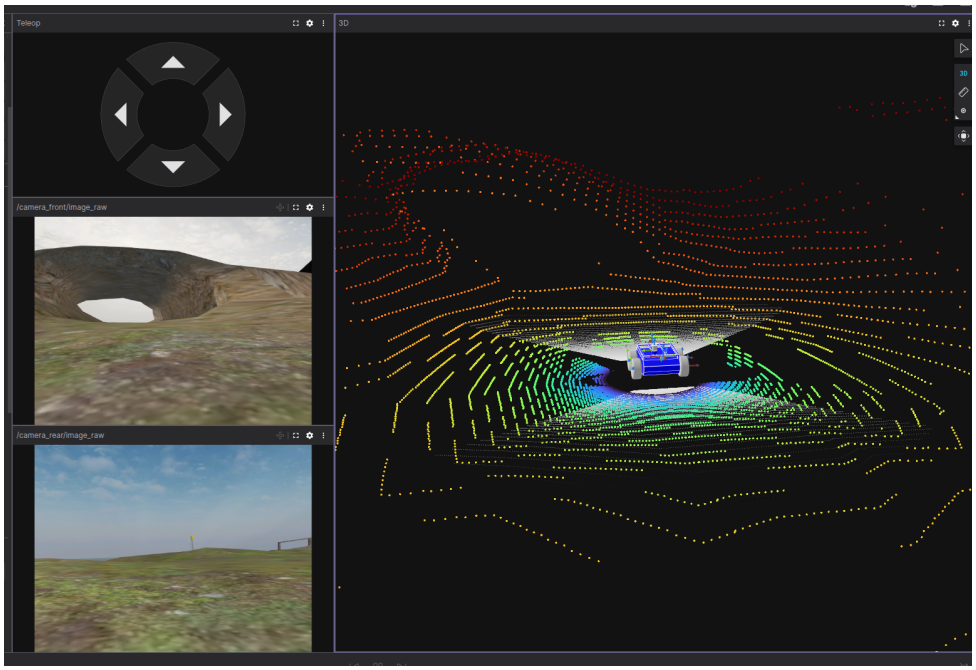
Listing 5.1: Przykładowe definicje przegubów w formacie xacro

5.3 Symulacja systemów percepcyjnych

Zintegrowany system percepcji wymagał odwzorowania charakterystyk sensorycznych robota w plikach konfiguracyjnych symulatora. W plikach `xacro` zdefiniowano parametry symulowanych sensorów, dopasowując je do ich fizycznych odpowiedników. Do weryfikacji poprawności akwizycji danych z symulatora wykorzystano oprogramowanie Foxglove, którego integracja z systemem ROS2 odbywa się na podstawie paczki przygotowanej przez producenta, `foxglove_bridge`.

5.3.1 Skaner laserowy (LIDAR 3D)

Parametry symulowanego skanera laserowego zostały zdefiniowane na podstawie specyfikacji technicznej czujnika Velodyne HDL32E. Narzędzie Foxglove potwierdziło poprawność działania sensora, m.in. na mapie `pipeline` (Rys. 5.4), gdzie chmura punktów poprawnie odwzorowywała ukształtowanie terenu oraz przeszkody (np. zarys wejścia do jaskini), co jest kluczowe dla algorytmów SLAM w środowisku terenowym.



Rysunek 5.4: Widok chmury punktów z symulowanego czujnika LIDAR na mapie pipeline.

```
<sensor name="laser" type="gpu_lidar">
    <pose> 0 0 0 0 0 0 </pose>
    <visualize>false</visualize>
    <update_rate>10</update_rate>
    <lidar>
        <scan>
            <horizontal>
                <samples>360</samples>
                <min_angle>-3.14</min_angle>
                <max_angle>3.14</max_angle>
            </horizontal>
            <vertical>
                <samples>32</samples>
                <resolution>1</resolution>
                <min_angle>-0.54</min_angle>
                <max_angle>0.18</max_angle>
            </vertical>
        </scan>
        <range>
            <min>0.3</min>
            <max>100.0</max>
        </range>
    </lidar>
    <topic>velodyne_scan</topic>
    <gz_frame_id>laser_frame</gz_frame_id>
    <visualize>true</visualize>
</sensor>
```

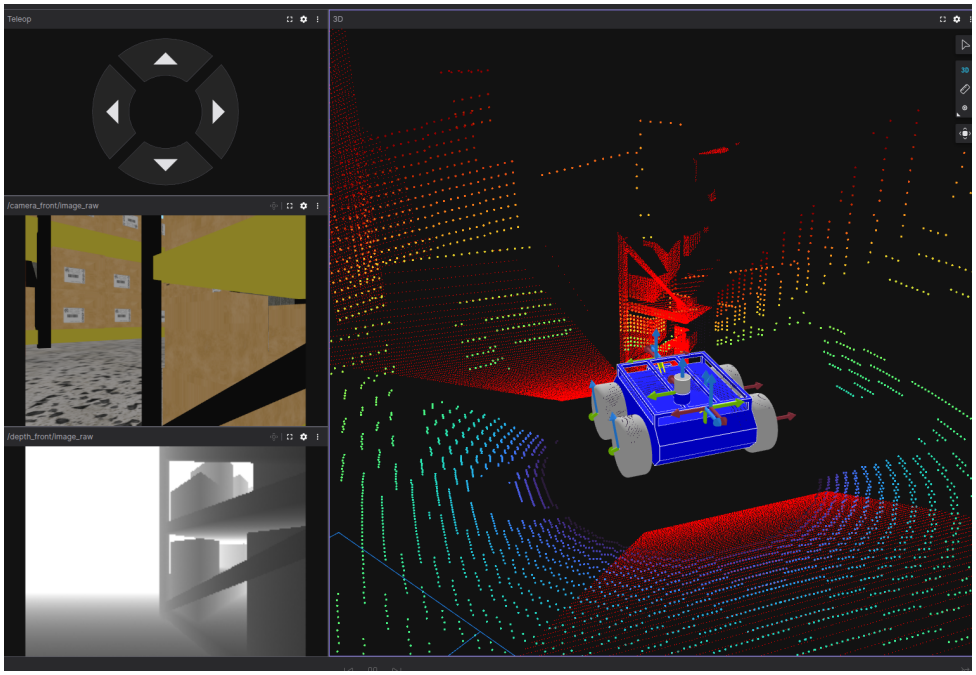
Listing 5.2: Definicja symulowanego skanera LIDAR 3D

5.3.2 Kamera RGB-D

Symulowana kamera ToF (Time of Flight) została sparametryzowana na wzór przemysłowych czujników O3R 225 firmy IFM. Zaimplementowano programowe dodawanie szumu pomiarowego (typu Gaussa), co pozwala na ewaluację algorytmów filtracji oraz badanie odporności algorytmów nawigacji na zakłócenia percepcyjne. Czujnik zwraca obraz kolorowy, macierz głębi (format 32FC1) oraz chmurę punktów, co zweryfikowano wizualnie (Rys. 5.5).

```
<sensor name="depth_rear" type="depth_camera">
    <always_on>1</always_on>
    <update_rate>20</update_rate>
    <visualize>0</visualize>
    <topic>depth_rear</topic>
    <pose>0 0 0 0 0 0</pose>
    <camera name="depth">
        <horizontal_fov>1.83</horizontal_fov>
        <vertical_fov>1.36</vertical_fov>
        <image>
            <width>224</width>
            <height>172</height>
            <format>RGB_INT8</format>
        </image>
        <clip>
            <near>0.1</near>
            <far>4</far>
        </clip>
        <noise>
            <type>gaussian</type>
            <mean>0</mean>
            <stddev>0.1</stddev>
        </noise>
    </camera>
    <gz_frame_id>rear_depth_frame</gz_frame_id>
</sensor>
```

Listing 5.3: Fragment definicji symulowanej kamery (czujnik głębi wraz z szumem)



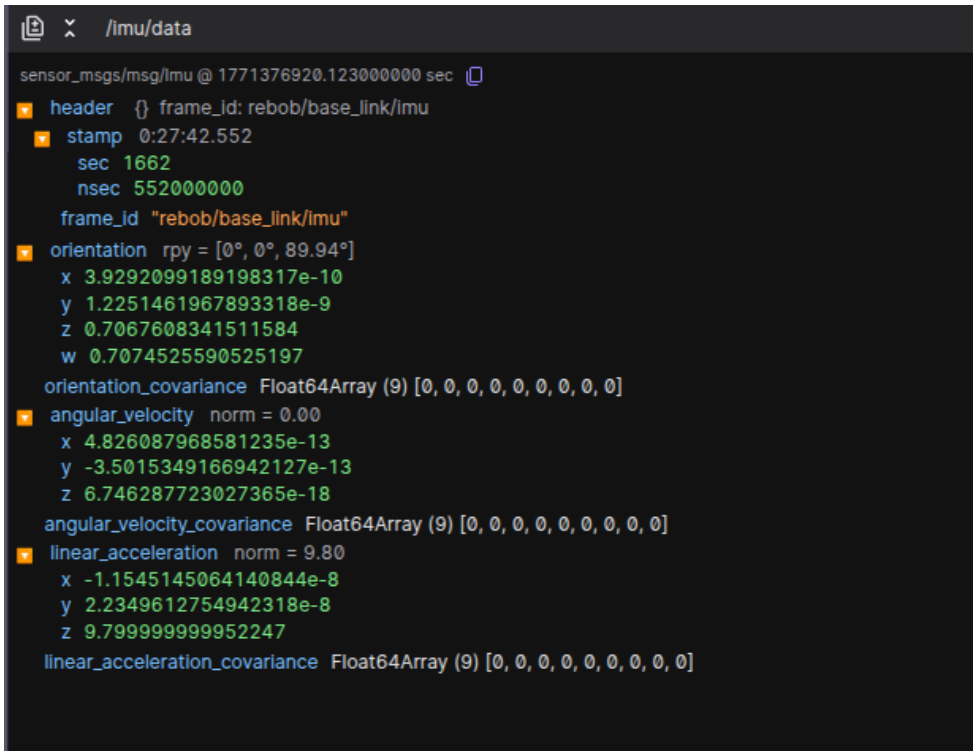
Rysunek 5.5: Akwizycja danych z symulowanej kamery RGB-D (obraz kolorowy, głębia, chmura punktów).

5.3.3 Inercyjna jednostka pomiarowa (IMU)

Moduł IMU dostarcza informacji o przyśpieszeniach liniowych i prędkościach kątowych obiektu. Na Rys. 5.6 zaprezentowano odczyty z symulowanego czujnika, które dla robota znajdującego się w spoczynku na płaskim podłożu wskazują wartości bliskie zeru (z uwzględnieniem wektora grawitacji), co potwierdza prawidłową inicjalizację sensora w środowisku Gazebo.

```
<sensor name="imu" type="imu">
    <always_on>1</always_on>
    <update_rate>20</update_rate>
    <visualize>true</visualize>
    <topic>imu</topic>
</sensor>
```

Listing 5.4: Definicja symulowanego czujnika IMU



Rysunek 5.6: Wizualizacja odczytów z symulowanego czujnika IMU.

Rozdział 6

Opracowanie zintegrowanego systemu percepcji

6.1 Integracja czujników

6.2 Przetwarzanie chmur punktów i filtracja danych

6.3 Implementacja metod 3D SLAM

6.4 Analiza przejezdności terenu

Rozdział 7

Konfiguracja i implementacja metod nawigacyjnych

7.1 Konfiguracja przestrzeni kosztów dla środowiska 3D

7.2 Implementacja metod nawigacyjnych

7.3 Adaptacja metod nawigacyjnych do platform testowych

Rozdział 8

Badania i analiza wyników

8.1 Metodyka badań i wskaźniki oceny wydajności

8.2 Eksperymenty w środowisku symulacyjnym

8.3 Testy rzeczywiste algorytmów na platformach testowych

8.4 Porównanie metod 3D SLAM

8.5 Porównanie algorytmów planowania ruchu

8.6 Wpływ kinematyki na skuteczność nawigacji terenowej

Rozdział 9

Podsumowanie i wnioski

9.1 Podsumowanie zrealizowanych prac

9.2 Ocena uzyskanych wyników

9.3 Perspektywy dalszego rozwoju

Bibliografia

[1] Abdulrahman Al-Batati, Anis Koubaa, Khaled Gabr, Mohamed Abdelkader i Hamad Aloqaily. „ROS 2 in a Nutshell: A Survey". W: *ACM Comput. Surv.* (maj 2026). Just Accepted. ISSN: 0360-0300. DOI: 10.1145/3815113. URL: https://doi.org/10.1145/3815113.

[2] Bashar Alsadik i Samer Karam. „The Simultaneous Localization and Mapping (SLAM)-An Overview". W: *Journal of Applied Science and Technology Trends* 2 (list. 2021), s. 120–131. DOI: 10.38094/jastt204117.

[3] Timothy D. Barfoot. *State Estimation for Robotics*. Cambridge University Press, 2017. DOI: 10.1017/9781316671528. URL: https://asrl.utias.utoronto.ca/~tdb/bib/barfoot_ser17.pdf.

[4] Behnam Behroozpour, Phillip A. M. Sandborn, Ming C. Wu i Bernhard E. Boser. „Lidar System Architectures and Circuits". W: *IEEE Communications Magazine* 55.10 (2017), s. 135–142. DOI: 10.1109/MCOM.2017.1700030.

[5] Paul J. Besl i Neil D. McKay. „A Method for Registration of 3-D Shapes". W: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), s. 239–256. DOI: 10.1109/34.121791. URL: https://doi.org/10.1109/34.121791.

[6] Jonas Beuchert, Marco Camurri i Maurice Fallon. „Factor Graph Fusion of Raw GNSS Sensing with IMU and Lidar for Precise Robot Localization without a Base Station". W: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2023, s. 8415–8421. DOI: 10.1109/ICRA48891.2023.10161522.

[7] Jose Luis Blanco-Claraco. „A flexible framework for accurate LiDAR odometry, map manipulation, and localization". W: *The International Journal of Robotics Research* 44.9 (2025), s. 1553–1599. DOI: 10.1177/02783649251316881. eprint: 2407.20465. URL: https://doi.org/10.1177/02783649251316881.

[8] Jose Luis Blanco-Claraco. „A Modular Optimization Framework for Localization and Mapping". W: *Proceedings of Robotics: Science and Systems*. 2019. DOI: 10.15607/RSS.2019.XV.043.

[9] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid i John J. Leonard. „Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age". W: *IEEE Transactions on Robotics* 32.6 (2016), s. 1309–1332. DOI: 10.1109/TRO.2016.2624754. URL: https://arxiv.org/abs/1606.05830.

[10] Duo Dong, Ziqi Wang, Jingyu Guan, Yi Xiao i Yuqi Wang. „Research on key technology and application progress of rescue robot in nuclear accident emergency situation". W: *Nuclear Engineering and Technology* 57.6 (2025), s. 103457. ISSN: 1738-5733. DOI: https://doi.org/10.1016/j.net.2025.103457. URL: https://www.sciencedirect.com/science/article/pii/S1738573325000257.

[11] Gregory Dudek i Michael Jenkin. „Inertial Sensors, GPS, and Odometry". W: *Springer Handbook of Robotics*. Red. Bruno Siciliano i Oussama Khatib. Berlin, Heidelberg: Springer-Verlag, 2008, s. 477–490. ISBN: 978-3-540-23957-4.

[12] Thomas Epton. „Odometry Correction of a Mobile Robot Using a Range-Finding Laser". Master's Thesis. Clemson University, grud. 2007. URL: https://cecas.clemson.edu/~ahoover/theses/epton-thesis.pdf.

[13] Sergi Foix, Guillem Alenyà i Carme Torras. „Lock-in Time-of-Flight (ToF) Cameras: A Survey". W: *IEEE Sensors Journal* 11.9 (2011), s. 1917–1926. DOI: 10.1109/JSEN.2010.2101060. URL: https://doi.org/10.1109/JSEN.2010.2101060.

[14] Christian Forster, Luca Carlone, Frank Dellaert i Davide Scaramuzza. „On-Manifold Preintegration for Real-Time Visual–Inertial Odometry". W: *IEEE Transactions on Robotics* 33.1 (2017), s. 1–21. DOI: 10.1109/TRO.2016.2597321. URL: https://dellaert.github.io/files/Forster16tro.pdf.

[15] Dieter Fox, Wolfram Burgard i Sebastian Thrun. „The Dynamic Window Approach to Collision Avoidance". W: *IEEE Robotics & Automation Magazine* 4.1 (1997), s. 23–33. DOI: 10.1109/100.580977. URL: https://publications.ri.cmu.edu/storage/publications/pub_files/pub1/fox_dieter_1997_1/fox_dieter_1997_1.pdf.

[16] Ian J. Griffiths, Qasim H. Mehdi, Tingkai Wang i Norman E. Gough. „A Genetic Algorithm for Path Planning". W: *IFAC Proceedings Volumes* 30.7 (1997). 3rd IFAC Symposium on Intelligent Components and Instruments For Control Applications 1997 (SICICA '97), Annecy, France, 9-11 June, s. 485–490. ISSN: 1474-6670. DOI: https://doi.org/10.1016/S1474-6670(17)43312-X. URL: https://www.sciencedirect.com/science/article/pii/S147466701743312X.

[17] Hui Guo, Zhaoyong Mao, Wenjun Ding i Peiliang Liu. „Optimal search path planning for unmanned surface vehicle based on an improved genetic algorithm". W: *Computers and Electrical Engineering* 79 (2019), s. 106467. ISSN: 0045-7906. DOI: https://doi.org/10.1016/j.compeleceng.2019.106467. URL: https://www.sciencedirect.com/science/article/pii/S0045790619310006.

[18] Peter E. Hart, Nils J. Nilsson i Bertram Raphael. „A Formal Basis for the Heuristic Determination of Minimum Cost Paths". W: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), s. 100–107. DOI: 10.1109/TSSC.1968.300136. URL: https://doi.org/10.1109/TSSC.1968.300136.

[19] Radu Horaud, Miles Hansard, Georgios Evangelidis i Clément Ménier. „An Overview of Depth Cameras and Range Scanners Based on Time-of-Flight Technologies". W: *Machine Vision and Applications* 27.7 (2016), s. 1005–1020. DOI: 10.1007/s00138-016-0784-4. URL: https://webspace.eecs.qmul.ac.uk/miles.hansard/papers/horaud-TOF.pdf.

[20] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss i Wolfram Burgard. „OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees". W: *Autonomous Robots* 34.3 (2013), s. 189–206. DOI: 10.1007/s10514-012-9321-0. URL: https://www.arminhornung.de/Research/pub/hornung13auro.pdf.

[21] Adeel Javaid. „Understanding Dijkstra Algorithm". W: *SSRN Electronic Journal* (sty. 2013). DOI: 10.2139/ssrn.2340905.

[22] Dena Kadhim, Firas Raheem i Ahmed Sadiq. „A Systematic Review of Rapidly Exploring Random Tree RRT Algorithm for Single and Multiple Robots". W: *Cybernetics and Information Technologies* 24 (wrz. 2024), s. 78–101. DOI: 10.2478/cait-2024-0026.

[23] Robert Lange i Peter Seitz. „Solid-State Time-of-Flight Range Camera". W: *IEEE Journal of Quantum Electronics* 37.3 (2001), s. 390–397. DOI: 10.1109/3.910448. URL: https://doi.org/10.1109/3.910448.

[24] Adarsh Nanjaiya Latha, Sabari Nathan Anbalagan, Alessandro Chiumento i Jan Laarhuis. „Where Do We Go from Here? Charting the Future of Unmanned Vehicles". W: *2025 16th IFIP Wireless and Mobile Networking Conference (WMNC)*. 2025, s. 176–181. DOI: 10.23919/WMNC67099.2025.11299247.

[25] Steven M. LaValle. *Planning Algorithms*. Cambridge University Press, 2006. URL: https://lavalle.pl/planning/.

[26] Xiangtai Li, Henghui Ding, Wenwei Zhang, Haobo Yuan, Guangliang Cheng, Pang Jiangmiao, Kai Chen, Ziwei Liu i Chen Change Loy. „Transformer-Based Visual Segmentation: A Survey". W: *T-PAMI* (2024).

[27] Yanan Liu, Yong Guan, Xiaojuan Li, Rui Wang i Jie Zhang. „Formal Analysis and Verification of DDS in ROS2". W: *2018 16th ACM/IEEE International Conference on Formal Methods and Models for System Design (MEMOCODE)*. 2018, s. 1–5. DOI: 10.1109/MEMCOD.2018.8556970.

[28] Jing Luo, Xiangyu Zhou, Chao Zeng, Yiming Jiang, Wen Qi, Kui Xiang, Muye Pang i Biwei Tang. „Robotics Perception and Control: Key Technologies and Applications". W: *Micromachines* 15.4 (2024). ISSN: 2072-666X. DOI: 10.3390/mi15040531. URL: https://www.mdpi.com/2072-666X/15/4/531.

[29] Steve Macenski, Shrijit Singh, Francisco Martín i Jonatan Ginés. „Regulated Pure Pursuit for Robot Path Tracking". W: *Autonomous Robots* 47.6 (2023), s. 685–694. DOI: 10.1007/s10514-023-10097-6. URL: https://arxiv.org/abs/2305.20026.

[30] Steven Macenski, Francisco Martin, Ruffin White i Jonatan Ginés Clavero. „The Marathon 2: A Navigation System". W: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020.

[31] Miguel Ángel de Miguel, Jorge Beltrán, Juan S. Cely, Francisco Martín, Juan Carlos Manzanares i Alberto García. *I Move Therefore I Learn: Experience-Based Traversability in Outdoor Robotics*. 2025. arXiv: 2507.00882 [cs.RO]. URL: https://arxiv.org/abs/2507.00882.

[32] Jauwairia Nasir, Fahad Islam i Yasar Ayaz. „Adaptive Rapidly-Exploring-Random-Tree-Star (RRT*) -Smart: Algorithm Characteristics and Behavior Analysis in Complex Environments". W: *Asia-Pacific Journal of Information Technology and Multimedia* 02 (grud. 2013), s. 39–51. DOI: 10.17576/apjitm-2013-0202-04.

[33] Nav2 Project. *Costmap 2D: Nav2 Documentation (Jazzy)*. URL: https://docs.nav2.org/jazzy/configuration_and_development/configuration_guide/core_servers/costmap_2d/ (term. wiz. 11. 10. 2026).

[34] Nav2 Project. *Inflation Layer Parameters: Nav2 Documentation (Jazzy)*. URL: https://docs.nav2.org/jazzy/configuration_and_development/configuration_guide/core_servers/costmap_2d/costmap_plugins/inflation/ (term. wiz. 12. 10. 2026).

[35] Nav2 Project. *Voxel Layer Parameters: Nav2 Documentation (Jazzy)*. URL: https://docs.nav2.org/jazzy/configuration_and_development/configuration_guide/core_servers/costmap_2d/costmap_plugins/voxel/ (term. wiz. 18. 10. 2026).

[36] Jiwei Qu, Zhe Zhang, Zheyu Qin, Kangquan Guo i Dan Li. „Applications of Autonomous Navigation Technologies for Unmanned Agricultural Tractors: A Review". W: *Machines* 12.4 (2024). ISSN: 2075-1702. DOI: 10.3390/machines12040218. URL: https://www.mdpi.com/2075-1702/12/4/218.

[37] R. H. Rasshofer, M. Spies i H. Spies. „Influences of Weather Phenomena on Automotive Laser Radar Systems". W: *Advances in Radio Science* 9 (2011), s. 49–60. DOI: 10.5194/ars-9-49-2011.

[38] Santiago Royo i Maria Ballesta-Garcia. „An Overview of Lidar Imaging Systems for Autonomous Vehicles". W: *Applied Sciences* 9.19 (2019), s. 4093. DOI: 10.3390/app9194093.

[39] Maulana Samodro, Riky Dp i Wahyu Caesarendra. „Artificial Potential Field Path Planning Algorithm in Differential Drive Mobile Robot Platform for Dynamic Environment". W: *International Journal of Robotics and Control Systems* 3 (mar. 2023), s. 161–170. DOI: 10.31763/ijrcs.v3i2.944.

[40] C. Saranya, Manju Unnikrishnan, S. Akbar Ali, D.S. Sheela i Dr. V.R. Lalithambika. „Terrain Based D* Algorithm for Path Planning". W: *IFAC-PapersOnLine* 49.1 (2016). 4th IFAC Conference on Advances in Control and Optimization of Dynamical Systems ACODS 2016, s. 178–182. ISSN: 2405-8963. DOI: https://doi.org/10.1016/j.ifacol.2016.03.049. URL: https://www.sciencedirect.com/science/article/pii/S2405896316300490.

[41] Andreas Serov, Joachim Clemens i Kerstin Schill. „Multi-Robot Graph SLAM Using LIDAR". W: *2024 10th International Conference on Automation, Robotics and Applications (ICARA)*. Athens, Greece, 2024. DOI: 10.1109/ICARA60736.2024.10553070.

[42] Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti i Daniela Rus. „LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping". W: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 2020, s. 5135–5142. DOI: 10.1109/IROS45743.2020.9341176.

[43] Tixiao Shan, Brendan Englot, Drew Meyers, Wei Wang, Carlo Ratti i Daniela Rus. „LIO-SAM: Tightly-Coupled Lidar Inertial Odometry via Smoothing and Mapping". W: *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, s. 5135–5142. DOI: 10.1109/IROS45743.2020.9341176. URL: https://arxiv.org/abs/2007.00258.

[44] Jarosław Szrek, Radoslaw Zimroz, Jacek Wodecki, Anna Michalak, Mateusz Góralczyk i Magdalena Worsa-Kozak. „Application of the Infrared Thermography and Unmanned Ground Vehicle for Rescue Action Support in Underground Mine—The AMICOS Project". W: *Remote Sensing* 13.1 (2021). ISSN: 2072-4292. DOI: 10.3390/rs13010069. URL: https://www.mdpi.com/2072-4292/13/1/69.

[45] David Tedaldi, Alberto Pretto i Emanuele Menegatti. „A Robust and Easy to Implement Method for IMU Calibration without External Equipments". W: *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, s. 3042–3049. DOI: 10.1109/ICRA.2014.6907297. URL: https://albertopretto.altervista.org/papers/tpm_icra2014.pdf.

[46] Zachary Teed i Jia Deng. „DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras". W: *Advances in neural information processing systems* (2021).

[47] Ananya Trivedi, Mark Zolotas, Adeeb Abbas, Sarvesh Prajapati, Salah Bazzi i Taskın Padır. *A Probabilistic Motion Model for Skid-Steer Wheeled Mobile Robot Navigation on Off-Road Terrains*. 2024. arXiv: 2402.18065 [cs.LG]. URL: https://arxiv.org/abs/2402.18065.

[48] Jasmin Velagic, Dada Delimustafic i Dinko Osmankovic. „Mobile Robot Navigation System Based on Probabilistic Road Map (PRM) with Halton Sampling of Configuration Space". W: czer. 2014. DOI: 10.1109/ISIE.2014.6864789.

[49] Haixia Wang, Shihao Wang i Tao Yu. „Path Planning of Inspection Robot Based on Improved Ant Colony Algorithm". W: *Applied Sciences* 14.20 (2024). ISSN: 2076-3417. DOI: 10.3390/app14209511. URL: https://www.mdpi.com/2076-3417/14/20/9511.

[50] Nan Wang, Xiang Li, Zhe Suo, Jiuchen Fan, Jixin Wang i Dongxuan Xie. „Traversability Analysis and Path Planning for Autonomous Wheeled Vehicles on Rigid Terrains". W: *Drones* 8.9 (2024). ISSN: 2504-446X. DOI: 10.3390/drones8090419. URL: https://www.mdpi.com/2504-446X/8/9/419.

[51] Hexiang Wei, Jianhao Jiao, Xiangcheng Hu, Jingwen Yu, Xupeng Xie, Jin Wu, Yilong Zhu, Yuxuan Liu, Lujia Wang i Ming Liu. „FusionPortableV2: A Unified Multi-Sensor Dataset for Generalized SLAM Across Diverse Platforms and Scalable Environments". W: *The International Journal of Robotics Research* 44.7 (2025), s. 1093–1116. DOI: 10.1177/02783649241303525.

[52] Greg Welch i Gary Bishop. *An Introduction to the Kalman Filter*. Spraw. tech. TR 95-041. Department of Computer Science, University of North Carolina at Chapel Hill, 2006. URL: https://d1.amobbs.com/bbs_upload782111/files_35/ourdev_611838G0F5Z1.pdf.

[53] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin i Fu Zhang. *FAST-LIO2: Fast Direct LiDAR-inertial Odometry*. 2021. DOI: 10.48550/arXiv.2107.06829. arXiv: 2107.06829 [cs.RO]. URL: https://arxiv.org/abs/2107.06829.

[54] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin i Fu Zhang. „FAST-LIO2: Fast Direct LiDAR-Inertial Odometry". W: *IEEE Transactions on Robotics* 38.4 (2022), s. 2053–2073. DOI: 10.1109/TRO.2022.3141876. URL: https://arxiv.org/abs/2107.06829.

[55] Wei Xu i Fu Zhang. „FAST-LIO: A Fast, Robust LiDAR-Inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter". W: *IEEE Robotics and Automation Letters* 6 (2020), s. 3317–3324. URL: https://api.semanticscholar.org/CorpusID:223953478.

[56] Yiqing Xu, Jiaming Li i Fuquan Zhang. „A UAV-Based Forest Fire Patrol Path Planning Strategy". W: *Forests* 13.11 (2022). ISSN: 1999-4907. DOI: 10.3390/f13111952. URL: https://www.mdpi.com/1999-4907/13/11/1952.

[57] Yumeng Yan. „Research on the A Star Algorithm for Finding Shortest Path". W: *Highlights in Science, Engineering and Technology* 46 (kw. 2023), s. 154–161. DOI: 10.54097/hset.v46i.7697.

[58] Zheng Zang, Xiaojie Gong, Xi Zhang i Jianwei Gong. „Coordinated path planning for autonomous ground vehicles in off-Road environments with 3D rigid terrain and obstacles". W: *Knowledge-Based Systems* 326 (2025), s. 114055. ISSN: 0950-7051. DOI: https://doi.org/10.1016/j.knosys.2025.114055. URL: https://www.sciencedirect.com/science/article/pii/S0950705125011001.

[59] Chenggong Zhai, Pinghua Zhang, Hongsi Xu, Xingguang Yuan, Liyong Zhou i Rangming Wu. „The Application and Inspiration of Robots in the US Military". W: *2023 8th Asia-Pacific Conference on Intelligent Robot Systems (ACIRS)*. 2023, s. 24–28. DOI: 10.1109/ACIRS58671.2023.10239710.

[60] Yuxiao Zhang, Alexander Carballo, Hanting Yang i Kazuya Takeda. „Perception and Sensing for Autonomous Vehicles Under Adverse Weather Conditions: A Survey". W: *ISPRS Journal of Photogrammetry and Remote Sensing* 196 (2023), s. 146–177. DOI: 10.1016/j.isprsjprs.2022.12.021.

Dodatki

Dokumentacja techniczna

Spis skrótów i symboli

Lista dodatkowych plików, uzupełniających tekst pracy (jeżeli dotyczy)

W systemie do pracy dołączono dodatkowe pliki zawierające:

- źródła programu,
- zbiory danych użyte w eksperymentach,
- film pokazujący działanie opracowanego oprogramowania lub zaprojektowanego i wykonanego urządzenia,
- itp.

Spis rysunków

5.1 Widok symulowanego robota w środowisku industrial warehouse. 30
5.2 Widok symulowanego robota w środowisku solar farm. 30
5.3 Widok symulowanego robota w środowisku pipeline. 31
5.4 Widok chmury punktów z symulowanego czujnika LIDAR na mapie pipeline. 34
5.5 Akwizycja danych z symulowanej kamery RGB-D (obraz kolorowy, głębia, chmura punktów). 37
5.6 Wizualizacja odczytów z symulowanego czujnika IMU. 38

Spis tabel